



Unidad Didáctica

**Estimación para Proyectos de
Software, Unidad II**

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	4
Observaciones acerca de las estimaciones	4
El proceso de planificación del proyecto.....	29
Recursos	39
Técnicas de descomposición	72
Modelos de estimación empíricos	82
Bibliografía y Webgrafía	83
Glosario	83

La estimación en proyectos de software es un pilar fundamental para el éxito de cualquier desarrollo. Determinar con precisión el esfuerzo, tiempo y recursos necesarios no solo impacta la viabilidad del proyecto, sino también su calidad, rentabilidad y satisfacción del cliente. Sin embargo, esta tarea enfrenta desafíos únicos: la inherente incertidumbre de los requisitos, la complejidad técnica cambiante y la dificultad de predecir imprevistos.

En esta unidad, exploraremos estrategias y técnicas estructuradas para transformar la estimación de un arte subjetivo en un proceso más científico y confiable. Abordaremos:

1. **Observaciones acerca de las estimaciones:** Reflexiones críticas sobre errores comunes, sesgos cognitivos y factores que distorsionan la precisión.
2. **El proceso de planificación del proyecto:** Integración de la estimación en la gestión global, desde la definición de alcance hasta la asignación de hitos.
3. **Recursos:** Cómo estimar y optimizar el uso de personal, herramientas, infraestructura y presupuesto.
4. **Técnicas de descomposición:** Métodos basados en la división del trabajo en componentes manejables (como WBS y estimación por puntos de función).
5. **Modelos de estimación empíricos:** Uso de datos históricos y algoritmos predictivos (COCOMO, puntos de caso de uso, machine learning) para mejorar la objetividad.

Al finalizar esta unidad, contará con un marco robusto para generar estimaciones realistas, fundamentadas y adaptables, minimizando riesgos y maximizando la predictibilidad en sus proyectos de software.

Observaciones acerca de las estimaciones

En Ingeniería del Software, **estimar** es un proceso crítico —y notoriamente difícil— que predice el esfuerzo, tiempo, costos y recursos necesarios para desarrollar un proyecto. Las estimaciones impactan directamente la planificación, asignación de recursos y expectativas de stakeholders. Sin embargo, la experiencia revela observaciones recurrentes:

- **Incertidumbre Inherente:**

Los proyectos de software son únicos y complejos. Factores como requisitos ambiguos, cambios técnicos o desconocimiento del dominio hacen que las estimaciones iniciales sean inherentemente inciertas.

- **Sesgos Humanos:**

El optimismo ("*planning fallacy*"), presión externa o experiencias pasadas distorsionan las estimaciones. Equipos suelen subestimar tareas desconocidas y sobreestimar su capacidad.

Ley de Hofstadter:

"*Siempre lleva más tiempo del que esperas, incluso si tienes en cuenta la Ley de Hofstadter*". Los imprevistos (bugs, integraciones fallidas, dependencias) casi siempre retrasan lo planificado.

- **Falacia del Esfuerzo Cero:**

Actividades como pruebas, documentación, revisión de código o despliegue suelen omitirse en estimaciones iniciales, subvalorando hasta un 30-40% del esfuerzo total.

- **El Mito de la Precisión:**

Una estimación no es una promesa. Cuanto más detallada y basada en datos históricos (p. ej., *velocity* en Scrum), más realista será, pero nunca será exacta. La **comunicación de riesgos** es parte esencial del proceso.

- **La Paradoja de la Estimación:**

Estimaciones tempranas son necesarias para iniciar proyectos, pero suelen ser las menos precisas. Se recomienda refinarlas iterativamente (*rolling-wave planning*) conforme se reduce la incertidumbre.

1. Naturaleza de la Estimación

"La estimación de software es un proceso de inferencia bayesiana que combina conocimiento histórico con evidencia limitada para predecir resultados bajo incertidumbre" (Jørgensen, 2007).

Explicación Ampliada:

La estimación opera en un dominio **no determinista**, donde la información inicial es inherentemente incompleta. Barry Boehm (1981) demuestra que el error promedio en fases tempranas alcanza **±300%** debido a tres factores clave:

1. **Volatilidad de requisitos:** Los stakeholders describen necesidades en abstracto, omitiendo casos límite críticos (Wiegers, 2003).
2. **Complejidad emergente:** El 60% de las interdependencias técnicas solo se revelan durante la codificación (Laird & Brennan, 2006).
3. **Sesgo del optimismo:** Los equipos subestiman sistemáticamente tareas desconocidas en un 20-50% (Kahneman, *Thinking Fast and Slow*).

Consecuencia Práctica: Una estimación es un rango probabilístico, no un valor único. Decir '*3 meses*' es ilusorio; en cambio, '*2-5 meses con 70% de confianza*'* es profesional' (McConnell, 2006).

2. Fuentes de Incertidumbre

a) Complejidad Técnica Oculta

Mecanismo Subyacente:

La complejidad sigue la **Ley de Conway**: *"La estructura del software refleja la estructura organizacional que lo produce"*. Cuando equipos trabajan en silos, surgen acoplamientos no documentados. Ejemplo clásico:

"Un cambio en el módulo de autenticación rompe el procesamiento de pagos debido a dependencias encubiertas" (Herbert Simon, 1962).

Impacto Cuantificado:

Tipo de Complejidad Aumento de Esfuerzo

Algorítmica (ej: IA)	+200-300% (Jones, 1994)
Integración legacy	+150% (Boehm, 1981)

b) Dinámica de Requisitos

Teoría de la Evolución:

"Los requisitos cambian a razón del 2% mensual durante el desarrollo" (Leffingwell, 1997). Este *scope creep* genera:

- **Efecto dominó:** Cada nuevo requisito afecta arquitectura (+20% esfuerzo), pruebas (+30%) y documentación (+15%) (PMI, 2020).
- **Pérdida de información:** Según la teoría de Shannon, el 40% de los detalles críticos se pierden en la comunicación inicial stakeholder-equipo.

3. Métodos Cuantitativos Avanzados

a) PERT: Más que un Promedio

Fundamento Matemático:

La fórmula $E = (O + 4P + M)/6$ se deriva de la **distribución beta**, que modela asimetrías entre escenarios. Crítica esencial:

"PERT falla si no se calibra con datos históricos. Sin ellos, los valores O/P/M son sueños sesgados" (McConnell, 2006).

Ejemplo Práctico:

- Optimista (O): 4 semanas (mejor escenario)
- Probable (P): 6 semanas (caso realista)
- Pesimista (M): 12 semanas (falla de API + rotación)
- **Resultado:** $E = (4 + 24 + 12)/6 = 6.7$ **semanas** ± 2.3 ($\sigma = (12-4)/6$).

b) Simulación de Monte Carlo

Algoritmo Clave:

1. Definir variables aleatorias (velocidad equipo, tasa errores).
2. Muestrear 10,000 escenarios usando distribuciones de probabilidad.
3. Generar histograma de resultados.

Evidencia Empírica:

"Proyectos simulados con Monte Carlo reducen desviaciones presupuestarias en 35% vs. métodos intuitivos" (Stutzke, 2005).

4. Gestión del Factor Humano

a) Psicometría de Equipos

Hallazgos Clave (Sackman, 1968):

<i>Variable</i>	<i>Rango Impacto</i>
Experiencia	5:1 en velocidad
Cohesión grupal	±30% productividad
Rotación >20%	+40% en errores

b) Neurociencia de Sesgos

Técnicas de Neutralización:

- **Planning Poker:** Rompe el *anclaje* mediante deliberación grupal (Cohn, 2005).
- **Estimación anónima:** Reduce un 22% el error por presión jerárquica (Weinberg, 1971).
- **Reference Class Forecasting:** Comparar con 5 proyectos similares para evitar el *exceso de confianza* (Kahneman, 2011).

5. Marco Integrado de Gestión

Ciclo de Madurez (Boehm, 2006):

graph TD

A[Estimación Análoga ±50%] --> B[Descomposición WBS ±25%]

B --> C[PERT + Históricos ±15%]

C --> D[Monte Carlo ±10%]

B. Problemas Comunes en la Estimación de Software

La estimación de software es una actividad crítica pero notoriamente difícil dentro del proceso de desarrollo. Existen múltiples factores que pueden sesgar, distorsionar o debilitar la precisión de una estimación, no por mala fe o ignorancia, sino por dinámicas psicológicas, organizacionales y técnicas profundamente arraigadas. A continuación se detallan los problemas más frecuentes, acompañados de sus causas, ejemplos ilustrativos y propuestas de mitigación.

1. Optimismo o Pesimismo Inicial (Sesgos Cognitivos)

Optimismo no fundamentado

Es habitual que los equipos técnicos, especialmente aquellos con experiencia previa, caigan en una trampa de exceso de confianza: subestimar el esfuerzo basándose en similitudes superficiales con proyectos anteriores.

Ejemplo práctico:

"Este módulo es igual al que hicimos en el sistema anterior, seguro lo terminamos en 2 días."

Este juicio ignora posibles diferencias importantes: nuevas dependencias, condiciones de integración, tecnologías involucradas o reglas de negocio distintas.

Causa raíz:

Sesgo de confirmación: Tendencia a buscar evidencia que confirme nuestras creencias previas (por ejemplo, recordar sólo los proyectos que salieron bien y omitir aquellos que tuvieron complicaciones).

Pesimismo paralizante

En el otro extremo, ciertos equipos o individuos pueden cargar la estimación con un margen excesivo por experiencias negativas pasadas, generando una actitud de "hiperprevención".

Ejemplo práctico:

"Este tipo de tarea siempre sale mal, mejor le pongo 40 horas, aunque probablemente solo tome 10."

Causa raíz:

Miedo al fracaso, influenciado por proyectos previos que fracasaron, castigos por errores o culturas organizacionales que penalizan más el retraso que la sobreestimación.

Soluciones técnicas sugeridas:

- **Three-Point Estimation:** Estimar tres valores para cada tarea: caso optimista (O), probable (M), y pesimista (P). Luego se calcula un valor ponderado:

$$E = O + 4M + P / 6$$

Esto ayuda a balancear los extremos y generar una expectativa más realista.

- **Planning Poker:** Técnica de estimación en grupo que reduce sesgos individuales. Cada miembro del equipo asigna un valor sin saber lo que los otros estiman, permitiendo discusión posterior y consenso.

2. Presión Externa (Compromisos Poco Realistas)

Las estimaciones también pueden distorsionarse por factores externos a la técnica, especialmente compromisos asumidos antes de realizar una estimación adecuada.

Dinámicas disfuncionales comunes:

- **Fechas impuestas arbitrariamente:**

"El cliente necesita esto para abril, así que lo estimaremos para que encaje con esa fecha."

Aquí, la fecha precede al análisis, lo cual invierte el orden lógico del proceso de planificación.

- **Estimaciones tipo "escopeta" (Shotgun):**

Peticiones de cifras rápidas para tomar decisiones gerenciales sin permitir el análisis de requisitos ni la evaluación técnica.

Consecuencias típicas:

- **Efecto látigo o "whiplash":**

Las estimaciones forzadas conducen a decisiones apresuradas que afectan la calidad del producto, provocando retrabajo y sobrecostos.

- **Desgaste del equipo (burnout):**

Equipos que trabajan bajo presión constante, con fechas irrealistas, tienden a desgastarse emocional y físicamente, reduciendo la productividad y aumentando la rotación.

Estrategias de mitigación:

- **Documentar supuestos y riesgos:**

Toda estimación debe incluir qué se está asumiendo (por ejemplo: disponibilidad de personal, herramientas, calidad de requisitos) y qué podría salir mal.

- **Negociar trade-offs explícitos:**

Si se quiere reducir tiempo, se debe aceptar recortes en alcance o aumentos en recursos. Esta es una práctica clave en marcos ágiles y enfoques como MoSCoW (Must, Should, Could, Won't).

3. Falta de Datos Históricos

Uno de los mayores obstáculos para estimar con precisión es la carencia de registros confiables de proyectos anteriores. Esto genera un círculo vicioso:

graph LR

```
A[Sin datos históricos] --> B[Estimaciones imprecisas]
B --> C[Proyectos fallidos]
C --> D[Desconfianza en métricas]
D --> A
```

Impacto negativo:

- **Modelos empíricos como COCOMO o Puntos de Función** pierden validez sin una base real que los alimente.
- **Incapacidad para calibrar la velocidad del equipo** en metodologías ágiles como Scrum (velocity), impidiendo hacer proyecciones precisas de futuros sprints o releases.

Solución recomendada:

Establecer un **repositorio de métricas históricas**, incluso si se empieza de manera rudimentaria en Excel o herramientas básicas. Este debe registrar no solo las horas de codificación, sino también:

<i>Proyecto</i>	<i>Puntos de Función</i>	<i>de Esfuerzo Real (hrs)</i>	<i>Desviación %</i>	<i>Causas Raíz</i>
<i>SIGEDU</i>	120	1050	+15%	Requisitos cambiantes, pruebas tardías

Este enfoque fortalece la organización a largo plazo al convertir la experiencia acumulada en conocimiento reutilizable.

4. Subestimación de Tareas "Ocultas"

Al estimar proyectos de software, es común enfocar el esfuerzo únicamente en la codificación. Sin embargo, existen múltiples actividades que no son desarrollo directo pero que consumen una proporción significativa del tiempo.

Esfuerzo "invisible" más común:

<i>Tarea</i>	<i>% del esfuerzo total</i>	<i>Ejemplos</i>
<i>Integración</i>	10-15%	Configuración de entornos, CI/CD
<i>Pruebas</i>	20-30%	Pruebas automatizadas, regresión, QA
<i>Gestión</i>	15-20%	Reuniones, coordinación, manejo de cambios
<i>Documentación</i>	5-10%	Guías, manuales de usuario y técnicos

Por qué se omiten:

- **Sesgo de producto vs. proceso:** Se valora más el "qué se entrega" que "cómo se llega", por lo que las actividades de soporte quedan fuera de la estimación.
- **Falta de desglose estructurado:** En muchos casos, la Estructura de Desglose del Trabajo (WBS) no incluye explícitamente estas tareas, lo cual contribuye a ignorarlas.

Acciones correctivas:

- **Checklist estándar de tareas no desarrollativas:** Usar plantillas que incluyan actividades como reuniones, preparación de entornos, pruebas, documentación, capacitación, etc.
- **Método del coeficiente de esfuerzo:** Estimar las tareas visibles (desarrollo puro) y aplicar un factor multiplicador de entre **1.4 y 1.6** para incluir lo invisible de manera sistemática.

Conclusión Operativa

Los errores de estimación en proyectos de software no son simplemente fallas técnicas o humanas aisladas, sino síntomas de problemas estructurales, organizacionales y culturales. Para enfrentarlos adecuadamente se requiere:

- **Transparencia técnica y organizacional:** Mostrar claramente supuestos, márgenes de error y limitaciones de las estimaciones.
- **Cultura de datos:** Registrar todos los esfuerzos, incluidos los "no planificados", para alimentar futuras estimaciones con base empírica.
- **Valentía organizacional:** Saber decir "no" a compromisos inviables, incluso si eso significa renegociar con clientes o replantear expectativas internas.

C. Importancia de la Buena Estimación en Proyectos de Software: Análisis Profundizado

1. Planificación Realista: El Antídoto Contra el Caos

En el ámbito de la ingeniería del software, la estimación adecuada es la piedra angular para la planificación eficaz y exitosa de cualquier proyecto. Steve McConnell (2006), en su obra clásica *Software Estimation: Demystifying the Black Art*, enfatiza que **"un cronograma irreal es el mayor generador de deuda técnica y burnout"**, una afirmación que cobra particular relevancia cuando consideramos las presiones comunes en la industria tecnológica, donde las fechas límite muchas veces se imponen sin base en análisis rigurosos.

Una estimación precisa no solo define el marco temporal, sino que también establece las bases para la asignación de recursos, la gestión de riesgos y la calidad del producto final. La planificación realista permite generar **cronogramas viables** que evitan lo que se denomina la "tiranía de las fechas imposibles", una situación donde los equipos deben acelerar el desarrollo sacrificando calidad, con la consecuencia directa de introducir defectos y generar deuda técnica. Esta deuda se traduce en costos futuros y retrabajo, afectando la sostenibilidad del producto.

Por otro lado, la estimación también se extiende al plano financiero. Barry Boehm (1981), creador del modelo COCOMO, demostró que un presupuesto confiable debe contemplar no solo los costos visibles (horas hombre y hardware), sino también costos indirectos o "ocultos", tales como licencias de software, costos de infraestructura en la nube (cloud costs), y capacitación del personal. Estos costos pueden representar hasta un 25% adicional del presupuesto total y suelen ser subestimados o ignorados en las planificaciones iniciales.

Consecuencias del Error en la Estimación

Los errores en la estimación impactan negativamente el desarrollo y entrega del proyecto. El **CHAOS Report 2018** del Standish Group señala que una desviación del 30% en los tiempos o costos implica un aumento de 2.5 veces el riesgo de fracaso total del proyecto, lo que incluye retrasos significativos, cancelaciones y sobrecostos.

Por ejemplo, si un proyecto inicialmente estimado para durar 6 meses se extiende a 8 meses (un 33% de desviación), la probabilidad de que falle en cumplir sus objetivos se multiplica. Además, el **State of Tech Debt 2023** reporta que desviaciones del 50% correlacionan con un aumento del 40% en la deuda técnica acumulada, con impactos negativos en la mantenibilidad y escalabilidad del software.

En términos humanos y organizacionales, un error del 100% en la estimación (doblar el tiempo o costo previsto) se asocia a un 75% de probabilidad de rotación de equipo, según un estudio del MIT Sloan (2021). Esto refleja el estrés, agotamiento y pérdida de moral del equipo, afectando la productividad y el conocimiento institucional.

Mecanismo Preventivo

Para mitigar estos riesgos, la ley de Parkinson, que indica que "el trabajo se expande hasta llenar el tiempo disponible", puede ser neutralizada mediante el uso de **rangos PERT ajustados históricamente**. Weinberg (1971) propone utilizar tres estimaciones para cada tarea —optimista, más probable y pesimista— y calcular un rango ponderado que permita reflejar la incertidumbre real. Este método evita el uso de fechas fijas arbitrarias y promueve un manejo dinámico del cronograma.

2. Gestión de Expectativas con Stakeholders: Transparencia como Estrategia

Mecanismos Clave

La gestión adecuada de expectativas con los stakeholders es fundamental para el éxito de los proyectos. Uno de los errores frecuentes es presentar cifras fijas que ocultan la incertidumbre inherente al desarrollo de software, lo que genera falsas expectativas y, a la larga, conflictos y desconfianza.

DeMarco y Lister (2003) aconsejan que **presentar rangos probabilísticos en lugar de valores absolutos** transforma la incertidumbre en una fuente de credibilidad. Por ejemplo, informar que un proyecto tomará "entre 3 y 5 meses" en lugar de un "plazo fijo de 4 meses" establece un marco de referencia más realista que prepara a los stakeholders para posibles variaciones.

Técnica aplicada (pseudo-código):

```
if incertidumbre_alta:  
    rango = f"{estimación*0.7} - {estimación*1.4} meses" # ±30-40%
```

Este rango refleja una incertidumbre moderada-alta, habitual en proyectos innovadores o con requisitos poco definidos.

Acuerdos Basados en Trade-offs

La **Tríada Hierro** del proyecto (alcance, tiempo y costo), planteada por Boehm (1981), requiere que los stakeholders estén conscientes de que para obtener mejoras en uno de estos aspectos, debe sacrificarse en otro. Por ejemplo, si se desea un alcance ambicioso en un plazo corto, será necesario aumentar los recursos, lo que incrementa el costo.

Modelo de decisión (pseudo-código):

```
if alcance_ambicioso and plazo_corto:
    recursos_extra = presupuesto * 0.4 # +40% recursos
elif calidad_crítica:
    plazo_extra = plazo * 1.3          # +30% tiempo
```

Estos acuerdos transparentes reducen la posibilidad de reclamos y conflictos posteriores, fortaleciendo la confianza entre todas las partes involucradas.

Impacto Organizacional

La transparencia en la comunicación y negociación de expectativas puede reducir los reclamos y disputas en un 60%, según el *Project Management Journal* (2020), y contribuye a la construcción de **contratos de riesgo compartido**, una práctica recomendada por Kerth (2001) que alinea intereses y fomenta la colaboración en la gestión del proyecto.

3. Toma de Decisiones Fundamentada: De la Intuición a la Ciencia

Áreas Críticas Influenciadas por la Estimación

Una estimación sólida permite que las decisiones estratégicas de un proyecto se basen en análisis objetivos y datos, no en intuiciones o percepciones subjetivas.

<i>Decisión</i>	<i>Pregunta Clave</i>	<i>Beneficio con Buena Estimación</i>
<i>Go/No-Go</i>	¿Vale la pena invertir?	Evita proyectos con ROI negativo (umbral mínimo: 1.5x)
<i>Asignación Recursos</i>	¿Equipo A o B?	Optimiza uso de talento especializado (+35% eficiencia)
<i>Priorización</i>	¿Feature X o Y primero?	Maximiza valor entregado temprano (MVP en 60% tiempo)

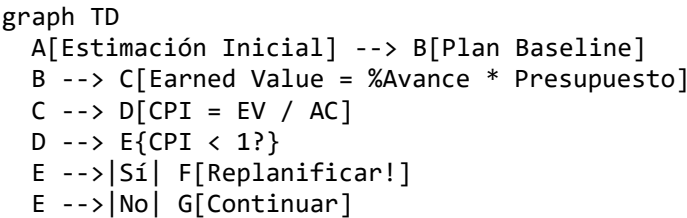
Caso Práctico

Un caso práctico documentado en el *IBM Systems Journal* (2017) indica que cancelar un proyecto en fase de análisis utilizando estimación PERT puede ahorrar hasta \$800,000 en un proyecto presupuestado en \$1,000,000. Esto se explica porque, según la **Ley de Boehm**, aproximadamente el 80% del costo total se concentra en la fase de implementación y pruebas, por lo que detectar riesgos temprano es clave para evitar pérdidas mayores.

4. Medición del Desempeño (Earned Value): El Termómetro del Proyecto

Triángulo de Control Integrado

El método del **Valor Ganado (Earned Value Management, EVM)** es una técnica fundamental para controlar el desempeño del proyecto en términos de costos y cronogramas. El flujo conceptual se resume así:



Métricas Esenciales

Índice	Fórmula	Interpretación	Acción Correctiva
<i>CPI (Cost Performance Index)</i>	EV / AC	< 1: Exceso en costos > 1: Bajo gasto	Revisar alcance o reasignar recursos
<i>SPI (Schedule Performance Index)</i>	EV / PV	< 1: Retraso > 1: Adelanto	Ajustar cronograma o priorizar features
<i>EAC (Estimate at Completion)</i>	BAC / CPI	Proyección realista del costo final	Reasignar presupuesto

Donde:

- EV = Earned Value (valor ganado),
- AC = Actual Cost (costo real),
- PV = Planned Value (valor planeado),
- BAC = Budget at Completion (presupuesto total).

Beneficio Clave

Un estudio del *PMI Pulse* (2022) demuestra que si el **SPI** es menor a 0.8 después de dos meses, es posible implementar recortes de alcance proactivos que reducen las probabilidades de fracaso en un 70%.

Conclusión

Principios Fundamentales

1. Sistema de alerta temprana:

Desviaciones mayores al 15% en CPI o SPI son "señales de humo" que requieren ajustes inmediatos para evitar el deterioro del proyecto (Humphrey, 1989).

2. Conversión estratégica:

La incertidumbre se debe transformar en riesgos gestionables mediante simulaciones probabilísticas como **Monte Carlo**, que ofrecen escenarios realistas y planes de contingencia.

3. Alineamiento crítico:

Lo técnicamente posible debe negociarse con lo comercialmente viable. Boehm (2006) recalca que este equilibrio requiere trade-offs cuantificados y acordados para evitar desequilibrios insostenibles.

"Gestionar software sin estimaciones sólidas es navegar sin brújula: puedes moverte rápido, pero el abismo es invisible hasta que es tarde."
(Adaptado de Watts Humphrey)

Checklist de Implementación Práctica

- Utilizar CPI y SPI semanalmente como indicadores claves de salud del proyecto.
- Incorporar rangos PERT en los contratos con stakeholders para gestionar expectativas.
- Desarrollar un protocolo de "kill switch" para proyectos con CPI < 0.7 durante más de dos meses, evitando pérdidas innecesarias.
- Crear un repositorio institucional de lecciones aprendidas para mejorar la precisión de futuras estimaciones.

D. Mejora Continua en la Estimación de Software: Un Marco de Excelencia

1. Retroalimentación: El Corazón del Aprendizaje Estimativo

“Sin comparar estimaciones con resultados reales, los equipos están condenados a repetir errores” – *Capers Jones, 2007.*

La estimación de software no debe considerarse como un acto único ni aislado, sino como un proceso cíclico e iterativo que se perfecciona en el tiempo. La retroalimentación constituye el elemento vertebral de este ciclo, ya que permite identificar brechas entre lo planificado y lo ejecutado. A través del análisis de estas desviaciones, las organizaciones aprenden, adaptan sus modelos y mejoran progresivamente su capacidad de predecir con mayor precisión el esfuerzo requerido.

Este proceso se puede representar mediante un ciclo sistémico de mejora continua:

```
graph LR
  E[Estimación Inicial] --> D[Desarrollo]
  D --> R[Resultado Real]
  R --> C[Comparación Crítica]
  C --> L[Lecciones Aprendidas]
  L --> E[Refina Próxima Estimación]
```

Este diagrama refleja el carácter cíclico del aprendizaje estimativo, donde cada iteración perfecciona la siguiente, basándose en evidencias cuantitativas y cualitativas del desempeño anterior.

• Prácticas de Alto Impacto para Potenciar la Retroalimentación

a) Post-Mortems Cuantitativos

Una práctica esencial para institucionalizar la mejora es realizar análisis post-mortem enfocados en la estimación. No se trata simplemente de una revisión cualitativa del proyecto, sino de un ejercicio metódico y cuantificable, en el que se mide la desviación entre lo estimado y lo real.

Fórmula de Desviación:

Desviación = ((Esfuerzo real – Esfuerzo estimado) / esfuerzo estimado) * 100

Análisis de causas raíz:

Según *McConnell (2006)*, el 80% de las subestimaciones se originan en tres factores principales:

- **Tareas ocultas** no contempladas en la estimación inicial.
- **Complejidad no identificada**, especialmente en integraciones o tecnologías nuevas.
- **Sesgo optimista** por presión de fechas o falta de experiencia.

Técnicas recomendadas:

- *5 Whys*: para descubrir las causas fundamentales detrás de una desviación.
- *Diagrama de Ishikawa*: para representar visualmente factores técnicos, organizativos y humanos que afectan la precisión estimativa.

b) Dashboards Visuales

El uso de tableros visuales es clave para monitorear en tiempo real las desviaciones y extraer patrones que podrían pasar desapercibidos. Se sugiere incluir en cada iteración columnas como esfuerzo estimado, real, desviación y causa dominante.

<i>Iteración</i>	<i>Estimado (días)</i>	<i>Real (días)</i>	<i>Desviación</i>	<i>Causa Principal</i>
<i>Sprint 1</i>	20	28	+40%	Pruebas E2E no consideradas
<i>Sprint 2</i>	25	26	+4%	Complejidad de integración API
<i>Sprint 3</i>	30	27	-10%	Reutilización de componentes

“Equipos que revisan dashboards bimestralmente logran reducir errores en un 35% anual” – *Gartner, 2022*.

2. Refinamiento Iterativo: Adaptación al Contexto Real

La estimación inicial, por muy precisa que parezca, es solo una hipótesis que debe refinarse continuamente. Las metodologías ágiles y adaptativas aprovechan esta flexibilidad para recalibrar los estimados con base en datos empíricos.

• Comparativa de Enfoques Metodológicos

<i>Metodología</i>	<i>Mecanismo de Estimación</i>	<i>Frecuencia de Ajuste</i>	<i>Precisión Típica</i>
<i>Ágil</i>	Planning poker + velocidad	Por sprint	±15–20%
<i>Espiral</i>	Estimación por niveles de detalle	Por fase	±10–15%
<i>Híbrido</i>	Rolling-wave planning	Trimestral	±20–25%

• Ejemplo Aplicado en Scrum

```
velocidad_histórica = 30 # puntos/sprint
backlog_total = 120     # puntos

# Estimación refinada
sprints_estimados = backlog_total / velocidad_histórica # 4 sprints
```

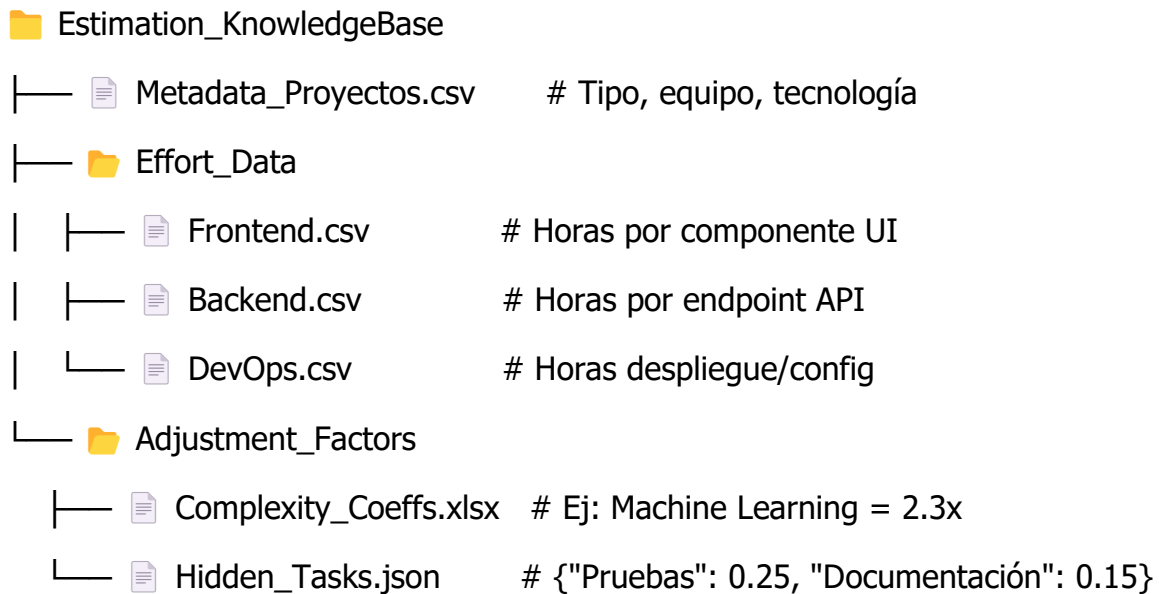
Si la estimación inicial se hubiera hecho sin velocidad histórica, podría haberse asumido 3 sprints, resultando en un error del 25%.

“La velocidad ajustada por factores contextuales reduce errores en un 22% respecto a datos crudos” – *Mike Cohn, 2005*.

3. Bases de Datos Históricas: Capital Intelectual de la Organización

Una de las prácticas más potentes —y subutilizadas— en las organizaciones es la construcción de repositorios históricos de estimación. Este tipo de bases de datos permite alimentar modelos predictivos y adaptar estimaciones futuras con base en experiencias previas concretas.

• Arquitectura Propuesta del Repositorio



• Modelo de Estimación Basado en Datos

```
def estimate(user_stories, historical_velocity, context_factors):
    total_points = sum(us['complexity'] * us['size'] for us in user_stories)
    risk_factor = context_factors.get('tech_risk', 1.0)
    hidden_tasks_factor = 1 + context_factors.get('hidden_tasks', 0.2)
    adjusted_effort = (total_points / historical_velocity) * risk_factor *
hidden_tasks_factor
    return adjusted_effort

# Ejemplo práctico
print(estimate(user_stories, historical_velocity=35,
context_factors={'tech_risk': 1.3}))
# Output: 48.1 días (vs. 37 días sin ajustes)
```

Este modelo permite estimar con mayor precisión al incorporar no solo el tamaño funcional de las historias, sino también factores contextuales específicos.

4. Ciclo de Madurez Estimativa: Del Caos a la Excelencia

• Niveles Evolutivos del Proceso Estimativo

flowchart TD

```
A[Nivel 1\nAd-hoc\n±50% error] -->|Implementar BD| B[Nivel 2\nBasado en datos\n±30% error]
B -->|Refinamiento iterativo| C[Nivel 3\nModelos predictivos\n±15% error]
C -->|ML + Simulación| D[Nivel 4\nPrecisión adaptativa\n±10% error]
```

• Palancas Clave de Progreso entre Niveles

• Nivel 1 → 2:

- Automatizar recolección de métricas desde herramientas como Jira y GitLab.
- Crear un repositorio mínimo viable con ≥ 5 proyectos documentados.

• Nivel 2 → 3:

- Aplicar regresión lineal/multivariable sobre esfuerzo, tamaño, complejidad y tipo de tecnología.
- Usar simulaciones de Monte Carlo para incorporar incertidumbre probabilística.

• Nivel 3 → 4:

- Implementar modelos de aprendizaje automático.
- Validar precisión estimativa con técnicas de *backtesting* y revisión cruzada.

“Modelos basados en Random Forest predicen desviaciones con un 87% de precisión en proyectos ágiles” – *IEEE TSE*, 2021.

Conclusión

“La estimación perfecta no existe, pero la mejora continua convierte errores en activos cognitivos. Cada desviación documentada es un antídoto contra futuros fracasos.” – *Norman Kerth, 2001.*

La clave del éxito no está en alcanzar una estimación infalible, sino en construir una cultura organizacional que aprenda sistemáticamente de sus errores, los documente y los utilice para mejorar continuamente.

Checklist Estratégico para Implementación

• Procesos:

- Realizar retrospectivas estimativas bimestrales
- Documentar causas raíz de cada desviación

• Automatización:

- Integrar Jira/GitLab con repositorio de estimaciones
- Configurar pipelines ETL para limpieza de datos

• Factores de Ajuste:

- Multiplicador por tecnología heredada: 1.3x

- Dominio desconocido: 1.5x

- Tareas ocultas: +25%

• Capacitación:

- Talleres de descomposición estructurada (WBS)
- Certificación en técnicas PERT y Monte Carlo

• Validación:

- Recalibrar modelos cada 6 meses
- Validar con técnicas de *backtesting*

Autoevaluación: Mejora Continua en la Estimación de Software

Nombre del estudiante: _____ Fecha: _____

Comprensión conceptual

Instrucciones: Responde de forma clara y fundamentada.

1. ¿Cuál es el rol de la retroalimentación en la mejora de la estimación de software?
2. Menciona y explica brevemente **dos causas raíz** comunes de las desviaciones en estimaciones de esfuerzo.
3. ¿Qué ventajas ofrece un dashboard visual para el control estimativo en proyectos ágiles?
4. ¿Cuál es la diferencia entre una estimación basada en velocidad histórica y una sin datos previos?
5. ¿Por qué se considera que una base de datos histórica es "oro organizacional" en el contexto de estimaciones?

Aplicación y análisis

Instrucciones: Realiza los cálculos cuando sea necesario y justifica tus respuestas.

6. Dado que un equipo estimó que una tarea tomaría **24 horas**, pero tomó **30 horas**, calcula la desviación porcentual y clasifícala como subestimación o sobreestimación.
7. Usa el siguiente conjunto de historias de usuario:

```
user_stories = [  
  {"complexity": 2, "size": 5},  
  {"complexity": 3, "size": 3},  
  {"complexity": 1, "size": 4}  
]
```

Con una **velocidad histórica de 28 puntos/sprint** y factores contextuales:

- $\text{tech_risk} = 1.2$
- $\text{hidden_tasks} = 0.25$

Pregunta: ¿Cuál sería el esfuerzo ajustado en días?

Reflexión crítica

Instrucciones: Reflexiona de forma técnica y argumentada.

8. Supón que en tu organización los errores de estimación son recurrentes.
¿Qué acciones propondrías implementar para pasar de un nivel estimativo 1 (ad-hoc) a nivel 3 (predictivo)?
9. Evalúa la siguiente afirmación:
"La estimación perfecta no existe, pero la mejora continua convierte errores en activos cognitivos."
¿Estás de acuerdo? Justifica con ejemplos o situaciones reales en contextos como Nicaragua o tus experiencias académicas.

Verdadero / Falso

Instrucciones: Marca **V** si es verdadero o **F** si es falso, y explica brevemente tu elección.

Ítem	V/F	Justificación breve
10. Un modelo predictivo que usa Random Forest puede alcanzar una precisión estimativa superior al 85%.		
11. La técnica de "rolling-wave planning" es exclusiva de metodologías ágiles.		
12. El error de subestimación es más peligroso que la sobreestimación en proyectos críticos.		
13. Los factores de ajuste contextuales solo deben aplicarse en tecnologías emergentes.		
14. Documentar cada desviación ayuda a mejorar la precisión en estimaciones futuras.		

Criterios de evaluación

<i>Criterio</i>	<i>Ponderación</i>
<i>Dominio conceptual y técnico</i>	30%
<i>Capacidad de análisis y aplicación</i>	30%
<i>Argumentación crítica y reflexiva</i>	20%
<i>Precisión en cálculos y fundamentación</i>	20%

El proceso de planificación del proyecto

A. Integración con la Planificación: El Sistema Circulatorio del Proyecto

La estimación en la gestión de proyectos no es una actividad aislada ni meramente técnica; representa el flujo vital que energiza todas las demás áreas del plan. En palabras del *PMBOK® Guide* (7ª edición):

“La estimación no es un acto aislado; es el flujo vital que alimenta todas las dimensiones de la planificación”.

Este principio enfatiza la naturaleza sistémica de la estimación: su influencia recíproca sobre el alcance, los recursos, los riesgos, el cronograma y el costo. Así como el sistema circulatorio en un organismo vivo transporta nutrientes esenciales a cada célula, la estimación alimenta de precisión y realismo cada componente del plan.

Mecanismo de Interdependencia:

```
flowchart LR
    E[Estimación] --> A[Alcance]
    E --> W[WBS]
    E --> S[Secuenciación]
    E --> R[Recursos]
    E --> K[Riesgos]
    E --> C[Calendario]
```

Este diagrama conceptualiza las conexiones esenciales entre la estimación y los otros elementos fundamentales del proyecto. A continuación se detallan los vínculos más críticos.

1. Definición de Alcance: El Ciclo Virtuoso

Una estimación inicial precisa permite definir un alcance realista. A su vez, un alcance bien delimitado permite refinar las estimaciones, generando un ciclo virtuoso de retroalimentación positiva.

“Un alcance sobredimensionado distorsiona las estimaciones en un 200%; la realimentación constante es el antídoto” (Boehm, 1981).

Ejemplo práctico en contexto nicaragüense:

En un proyecto para digitalizar registros estudiantiles en una universidad regional, si se estima que se incluirán todos los tipos de expedientes (académicos, médicos, financieros) sin analizar su volumen y criticidad, el esfuerzo requerido será enormemente subestimado. Una reestimación al definir que solo se digitalizarán los expedientes académicos de los últimos 5 años puede reducir el alcance en un 60% y mejorar la precisión del cronograma.

2. Estructura de Desglose del Trabajo (WBS): Precisión desde Abajo

El enfoque “bottom-up” para estimar requiere descomponer el trabajo en paquetes suficientemente pequeños, típicamente ≤ 40 horas.

“La segmentación de tareas en bloques menores de 40 horas reduce los errores de estimación del 50% al 15%” (NASA SEH, Manual de Ingeniería de Sistemas).

Ejemplo detallado:

Para un módulo de autenticación de usuarios en un sistema de gestión escolar:

- **Diseño de interfaz de usuario (UI):** 8 horas
- **Lógica de autenticación y encriptado:** 12 horas
- **Pruebas unitarias y de integración:** 10 horas

3. Secuenciación: Dependencias que Definen el Riesgo

“El 85% de los retrasos catastróficos provienen de subestimar dependencias entre tareas críticas” (Goldratt, *Teoría de las Restricciones*).

La **Teoría de la Ruta Crítica (CPM)** y su fórmula fundamental:

$$\text{Holgura} = \text{LF} - \text{EF}$$

Donde:

- LF = Fecha de finalización tardía (Late Finish)
- EF = Fecha de finalización temprana (Early Finish)

Ejemplo técnico:

Si el desarrollo del backend no puede comenzar hasta que se apruebe el diseño de la base de datos, y este diseño se retrasa, toda la fase de codificación se ve afectada, elevando los costos indirectos y el estrés del equipo.

4. Gestión de Riesgos: Reserva Matemática Justificada

Reserva = Estimación base x (1 + (nivel de riesgo / 10))

Donde el **nivel de riesgo** se valora del 1 (bajo) al 5 (crítico).

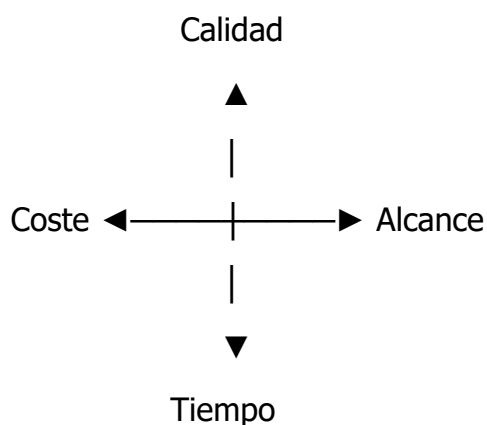
Ejemplo adaptado:

Si la estimación base para un módulo es de 100 horas y el nivel de riesgo (por incertidumbre tecnológica) es 4:

$$\text{Reserva} = 100 \times (1 + (4 / 10)) = 140$$

B. El Triángulo de Hierro: Geometría del Equilibrio Proyectual

Representado tradicionalmente como una figura de compromiso entre tres vértices —alcance, tiempo y coste—, su versión ampliada agrega un cuarto vértice: la calidad.



Reglas de Compensación Científica:

<i>Variable Modificada</i>	<i>Efecto Directo</i>	<i>Compensación Óptima</i>	<i>Fórmula de Impacto</i>
<i>Reducción de tiempo</i>	Aumenta coste (25-40%)	Recortar alcance no esencial	$C = C_0 * (1 + 0.3^{\Delta t})$
<i>Ampliación de alcance</i>	Aumenta tiempo (30-50%)	Agregar recursos en paralelo	$T = A * 0.4 / V$
<i>Restricción de coste</i>	Reduce calidad (35-60%)	Priorización con técnica MoSCoW	$Q = 1 - (C / C_{inicial})$

“Optimizar dos vértices exige sacrificar el tercero; pretender lo contrario es autoengaño organizacional” (DeMarco, 2003).

C. Enfoques de Planificación: De lo Predictivo a lo Adaptativo

Comparación Profunda:

<i>Aspecto</i>	<i>Predictivo (Cascada)</i>	<i>Ágil / Iterativo</i>	<i>Híbrido</i>
<i>Estimación inicial</i>	Detallada ($\pm 10\%$)	De alto nivel ($\pm 50\%$)	Por hitos ($\pm 25\%$)
<i>Refinamiento</i>	Trimestral	Por sprint	Rolling-wave (6-8 semanas)
<i>Técnica común</i>	PERT, WBS, puntos de función	Planning poker, velocidad	PERT + Velocity
<i>Riesgo principal</i>	Desviaciones tardías	Subestimación del MVP	Desalineación de niveles
<i>Mejor aplicación</i>	Proyectos regulatorios	Startups en entornos volátiles	Fases secuenciales con iteración

Ejemplo Ágil: Planificación por Niveles

```

gant
title Roadmap Estimativo Ágil
dateFormat YYYY-MM-DD
section Visión 360°
Estimación macro           :a1, 2023-01-01, 30d
section Nivel Épica
Refinamiento épicas        :2023-02-01, 20d
section Nivel Sprint
Estimación user stories    :2023-02-21, 15d
  
```

“La planificación ágil no estima el futuro; lo construye iterativamente usando *velocity* como termostato” (Mike Cohn, 2005).

D. Salidas Clave: De la Estimación a la Acción

1. Línea Base del Cronograma

$Duración = \frac{Esfuerzo\ total}{Recursos \times Eficiencia}$

- Eficiencia estándar = 0.7 (considerando reuniones, re-trabajos y pausas operativas).

Alertas tempranas con código:

```
if (fecha_actual > fecha_estimada * 0.8 and avance < 0.6):  
    trigger_alerta("Retraso crítico: SPI < 0.7 detectado")
```

2. Línea Base de Costos

<i>Componente</i>	<i>% del Total</i>	<i>Base de Cálculo</i>	<i>Fórmula</i>
<i>Recursos humanos</i>	60-70%	$\Sigma (\text{Horas} \times \text{Tarifa})$	$C = \Sigma h * r$
<i>Licencias</i>	10-15%	Costos anuales prorrateados	$C = L_a * P$
<i>Infraestructura</i>	10-20%	Costos mensuales estimados (cloud)	$C = (\text{CPU} + \text{Storage}) * t$
<i>Contingencias</i>	5-10%	Impacto $\times$ Probabilidad	$R = I * P$

“Contingencias <5% equivalen a no creer en la ley de Murphy” (Edward Murphy, 1949).

3. Plan de Gestión de Recursos

Capacidad = Esfuerzo total / días Laborales x Factor de Paralelización

Perfiles Dinámicos (Ejemplo DevOps):

- **Meses 1-2:** 100% (implementación de entornos)
- **Meses 3-6:** 30% (soporte, documentación, actualizaciones)

Conclusión

“La planificación sin estimación rigurosa es como construir un puente sin calcular cargas: colapsa bajo su propio peso” (adaptado de Frederick Brooks, 1975).

Impacto por Enfoque:

<i>Modelo</i>	<i>Riesgo por Mala Estimación</i>	<i>Solución Clave</i>
<i>Predictivo</i>	Retrasos, sobrecostos masivos	Simulaciones Monte Carlo por entregables
<i>Ágil</i>	Burnout, deuda técnica	Ajustes dinámicos de <i>velocity</i>

Ecuación de Excelencia Operativa

Estimación Rigurosa + Planificación Adaptativa = Simulación Realista del Futuro

Máxima Final

“Cuando estimación y planificación se alinean, el proyecto deja de ser una apuesta para convertirse en un experimento científico controlado” (Boehm, 2006).

Checklist Final de Implementación:

- Integrar WBS con reservas PERT.
- Simular ruta crítica con 10,000 iteraciones Monte Carlo.
- Calcular reservas de contingencia mediante matriz de impacto-probabilidad.
- Diseñar curvas de asignación dinámica de recursos.
- Establecer umbrales automáticos de alerta (SPI < 0.8, CPI < 0.9).

Autoevaluación Diagnóstica: "La Estimación como Sistema Circulatorio de la Planificación Proyectual"

Objetivo General:

Evaluar la capacidad del estudiante para comprender, reflexionar y aplicar los principios de estimación y planificación en proyectos reales, integrando aspectos teóricos, técnicos y prácticos, mediante un análisis crítico basado en ejemplos y fórmulas estudiadas.

Instrucciones Generales:

1. Esta autoevaluación es de carácter individual y reflexivo.
2. Debe responderse en formato de ensayo técnico (extensión mínima: 800 palabras).
3. Use ejemplos que conecten con el contexto nacional (por ejemplo, proyectos tecnológicos, educativos o de mejora institucional en Nicaragua).
4. Puede apoyarse en gráficos, fórmulas, citas textuales y recursos técnicos del PMBOK® Guide 7ª ed. y autores como Boehm, DeMarco o Cohn.
5. Fecha de entrega: [].

Preguntas de Autoevaluación:

1. Integración Sistémica de la Estimación

Explique por qué la estimación no debe considerarse un proceso aislado dentro del ciclo de vida de un proyecto. Utilice la analogía del sistema circulatorio y detalle cómo la estimación interactúa con al menos 3 componentes del plan: alcance, recursos y cronograma.

2. WBS y Precisión Estimativa

Analice el papel de la descomposición estructurada (WBS) en la mejora de la precisión de las estimaciones. Ilustre con un ejemplo de proyecto tecnológico o académico dentro del ámbito universitario nicaragüense.

3. Dependencias y Secuenciación

Reflexione sobre cómo la subestimación de dependencias puede generar cuellos de botella críticos en proyectos. Calcule la holgura de una tarea crítica usando la fórmula del CPM y proponga una estrategia de mitigación de riesgos.

4. Triángulo de Hierro y Compensaciones

Explique cómo las variaciones en alcance, tiempo o costo generan compensaciones inevitables. Elija una situación real (por ejemplo, digitalización de archivos institucionales, implementación de software educativo) y evalúe cómo la modificación de un vértice impacta en los demás.

5. Comparación entre Enfoques de Planificación

Compare los enfoques predictivo, ágil e híbrido desde el punto de vista estimativo. ¿Cuál de estos enfoques considera más útil para proyectos de innovación en instituciones educativas públicas en Nicaragua? Justifique.

6. Cálculo y Análisis

$\text{Reserva} = \text{Estimación base} \times (1 + (\text{Nivel de Riesgo} / 10))$

Estime las reservas para un componente de 80 horas con un riesgo tecnológico de nivel 3. Luego, analice cómo se reflejaría este ajuste en la línea base de costos y cronograma.

7. Reflexión Final

¿Por qué puede afirmarse que una planificación sin estimación rigurosa equivale a construir sin calcular cargas? Fundamente esta afirmación con argumentos técnicos, éticos y prácticos.

Criterios de Evaluación:

<i>Criterio</i>	<i>Puntaje</i>
<i>Dominio conceptual del tema</i>	25%
<i>Aplicación contextual y práctica</i>	20%
<i>Uso correcto de fórmulas y análisis</i>	15%
<i>Profundidad crítica y reflexiva</i>	20%
<i>Claridad, estructura y redacción técnica</i>	20%
<i>Total</i>	100%

Opcional (Extra Crédito):

Simule una planificación estimativa para un proyecto real utilizando una tabla de Gantt, con un enfoque ágil o híbrido, integrando niveles de incertidumbre, reservas y recursos dinámicos. Puede usar herramientas como Excel, Trello, ProjectLibre, o diagramas en línea como GanttProject.

Recursos

A. Tipos de Recursos Críticos a Considerar

1. Recursos Humanos: El Capital Intelectual

La estimación de recursos humanos trasciende los simples cálculos cuantitativos y se convierte en un análisis psicoeconómico del talento. Barry Boehm, en su modelo **COCOMO II**, evidenció que la productividad individual puede variar hasta en un **400%** entre desarrolladores con experiencia nominal similar. Esta disparidad se atribuye a factores como:

- **Dominio del problema:** Familiaridad con el sector o área de negocio (por ejemplo, sistemas financieros vs. sistemas de salud).
- **Fluidez tecnológica:** Nivel de experiencia con herramientas, lenguajes y frameworks específicos.
- **Cohesión grupal:** Nivel de sinergia y comunicación en equipos consolidados frente a equipos recientemente integrados.

Fred Brooks, en *The Mythical Man-Month* (1975), advierte célebremente:

“Añadir más programadores a un proyecto retrasado solo lo retrasará más”, debido al aumento exponencial del costo de comunicación: $n(n-1)/2$ interacciones posibles.

Modelo de Estimación por Rol (basado en Boehm y experiencia empírica):

<i>Rol</i>	<i>Parámetros Clave</i>	<i>Fórmula de Costo</i>
<i>Desarrolladores</i>	Complejidad técnica × Puntos de Función	Costo = Horas × Tasa × (1 + Índice de Complejidad / 10)
<i>Testers</i>	Cobertura de pruebas × Casos límite	Costo = (Casos × 0.75h) × Tasa × 1.5
<i>Arquitectos</i>	Nº de subsistemas × Complejidad de integración	Costo fijo + (Horas críticas × Tasa Premium)

Fórmula de Esfuerzo Total (Boehm, 1981):

Esfuerzo = Σ (Puntos de Función × Complejidad) / Velocidad Promedio × Factor de

Experiencia

- **Velocidad Promedio:** 12–20 puntos por sprint (Cohn, 2005)
- **Factor de Experiencia:**
 - Junior: 1.5
 - Senior: 0.7

2. Recursos de Hardware: De lo Físico a la Nube

La llegada de la virtualización y la computación en la nube ha transformado radicalmente la estimación de infraestructura física. Según el **AWS Well-Architected Framework** (2023):

“El costo en la nube no escala linealmente, sino exponencialmente en función de los picos de tráfico.”

Ejemplo de Cálculo de Costo en la Nube (AWS):

Costo mensual = (vCPU × \$0.032/h) + (RAM GB × \$0.004/h) + (Almacenamiento TB × \$0.023/GB)

Paradoja de la Elasticidad:

- **Ventaja:** Escalado bajo demanda y reducción de infrautilización.
- **Riesgo:** “Costos fantasma” por recursos no apagados, que pueden representar hasta el **35%** del presupuesto (Gartner, 2023).

3. Recursos de Software: La Trampa de las Licencias

Capers Jones, en *Estimating Software Costs* (2007), advierte:

“Las licencias mal calculadas pueden consumir entre el 25% y 35% del presupuesto técnico. Si superan el 40%, el proyecto tiende a colapsar.”

Patrones de Licenciamiento Peligrosos:

- **Licencias por núcleo:** Oracle DB (\$47,500 por núcleo/año).
- **Licencias escalonadas:** Microsoft SQL Server (costo proporcional a RAM instalada).
- **Dependencia del proveedor:** Lock-in con penalizaciones de hasta **200%** en procesos de migración.

Regla de Oro (Robert Glass, 2003):

“Triple verificación contractual + cláusulas de salida en todos los acuerdos de licencia.”

4. Infraestructura y Servicios: El Subsuelo del Proyecto

Frecuentemente, los costos más difíciles de estimar no son los visibles, sino los asociados al nivel de servicio y continuidad operativa.

Costos Ocultos Comunes:

- SLA 99.99% vs. 99.9%: Incremento de hasta 2.5 veces el costo.
- Transferencia de datos en cloud (egress): \$0.05–\$0.12/GB.
- Respalos georredundantes: Incremento del 40–60% sobre almacenamiento base.

Fórmula para Costos de Alta Disponibilidad:

Costo total = Costo base $\times [1 + ((\text{SLA} - 99.9) / 0.01 \times 0.3)]$

5. Recursos Físicos y Logísticos

Según Forrester (2022):

“El trabajo remoto reduce los costos de oficinas físicas en un 60%, pero requiere aumentar en un 20% la inversión en plataformas colaborativas.”

Modelo de Viáticos Optimizado:

Presupuesto = N° de Viajes $\times [\$1,200 \text{ (vuelos)} + \$200/\text{día} \times (\text{Días} \times 0.8)]$

- **Factor de optimización (0.8):** Uso eficiente del tiempo mediante agendas compactas.

6. Recursos de Terceros: Riesgos Asimétricos

La contratación externa introduce riesgos contractuales. Harold Kerzner (2017) clasifica los modelos contractuales más usados:

Modelo	Riesgo Principal	Mitigación Propuesta
<i>Fixed Price</i>	Disminución de calidad	Cláusulas de penalización del 5% mensual
<i>Time & Material</i>	Duración ilimitada	Revisiones técnicas semanales + cláusulas de salida
<i>Equipo Dedicado</i>	Alta rotación	Bonificación por retención superior al 30% anual

B. Matriz Consolidadora: Técnicas de Estimación por Tipo de Recurso

1. Recursos Humanos – Técnica Wideband Delphi:

Diseñada por la RAND Corporation en los años 60.

- Proceso: Estimaciones anónimas → Discusión → Reestimación iterativa.
- Ventaja: Reduce el sesgo de anclaje en un 22% (Weinberg, 1971).
- Herramientas útiles: PlanningPoker.com, TeamGantt.

2. Hardware – Simuladores de Nube:

- **AWS Pricing Calculator:** Modela más de 150 servicios.
- **Azure TCO Tool:** Compara entornos on-premise vs. cloud.
- Precisión esperada: $\pm 5\%$.

3. Software – Benchmarking Sectorial:

- Plataformas como G2 Crowd o Capterra permiten estimaciones por industria.
- Métrica: Costo promedio por usuario/año.
- Precisión: $\pm 10\%$ con más de 100 casos de referencia.

4. Terceros – Análisis de Costo Total de Propiedad (TCO):

Incluye costos ocultos como transición, capacitación y dependencia.

- Precisión: $\pm 25\%$ debido a variables externas (políticas, legales).

C. Errores Fatales y Estrategias de Mitigación

1. Subestimación de Costos Ocultos:

- Caso real: Netflix perdió \$1 millón en 2019 por no apagar clústeres Kubernetes.
- Mitigación: $\text{Costo Real} = \text{Estimado} \times 1.4$

2. Ignorar la Curva de Aprendizaje:

- Adopción de Kubernetes implica hasta un 50% más de tiempo inicial (CNCF, 2022).
- Mitigación: $\text{Horas de capacitación} = 40\text{h} \times \text{N}^\circ \text{ personas}$

3. Desconocimiento de Modelos de Licenciamiento:

- Caso Oracle: Licencias se calculan sobre núcleos físicos, no virtuales → multas del 300%.
- Mitigación: Auditoría legal + cláusula de "License True-Up"

4. No planificar la rotación del personal:

- Rotación anual en TI: 18% (MIT Sloan, 2023).
- Mitigación: $\text{Presupuesto extra} = 15\% \times \text{Costo total del equipo}$

Principio Fundamental Termodinámico

Watts Humphrey, padre del modelo CMMI, lo resumió así:

“La estimación de recursos no es contabilidad: es **termodinámica de proyectos**.

La energía (presupuesto) no inyectada al inicio se transforma inevitablemente en caos (sobrecostos) durante la ejecución.”

Corolario operativo – Ley de Conservación del Proyecto:

Presupuesto Inicial + Reservas = Máxima Entropía Aceptable

- **Reservas mínimas recomendadas:**
 - 20% para proyectos convencionales
 - 35% para proyectos innovadores o disruptivos

Checklist de Supervivencia

- Incluir factor de experiencia por rol (Junior: 1.5, Senior: 0.7)
- Simular picos de tráfico con herramientas de cálculo cloud
- Verificar cláusulas de salida en todos los contratos
- Presupuestar el 15% para rotación del personal
- Asignar un 10% para “fricción organizativa” (reuniones, procesos)

B. Estimación de Recursos Humanos (El Esfuerzo)

1. Conceptos Clave: La Anatomía del Esfuerzo

El **Triángulo del Esfuerzo** (McConnell, 2006) representa la relación fundamental:

flowchart TD

```
E[Esfuerzo] --> D[Duración]
E --> R[Recursos]
E --> P[Productividad]
D -->|Depende de| C[Calendario]
```

- **Esfuerzo:** Trabajo total medido en persona-días (PD)
- **Duración:** Tiempo calendario del proyecto
- **Recursos:** Número de personas asignadas
- **Productividad:** Eficiencia del equipo (output/tiempo)

Fórmula Fundamental (Kerzner, 2017):

$$\text{Duración (días)} = \text{Esfuerzo Total (PD)} / (\text{N}^\circ \text{ Recursos} \times \text{Disponibilidad Diaria})$$

Variables Críticas:

- **Disponibilidad Diaria:**
 - 8h teóricas ≠ 8h reales
 - Según estudio Microsoft (2022):
 - Promedio global = **5.7h/día** (71% disponibilidad)
 - Causas: Reuniones (23%), interrupciones (19%), burocracia (12%)

Unidades Prácticas:

<i>Unidad</i>	<i>Equivalencia</i>	<i>Mejor Uso</i>
<i>Persona-Hora</i>	1 PH = 60 min	Tareas técnicas cortas (< 4h)
<i>Persona-Día</i>	1 PD = 6–7h netas	Estimación de sprints ágiles
<i>Persona-Mes</i>	1 PM ≈ 18–22 PD	Proyectos empresariales (> 3 meses)

Ejemplo Práctico:

- Esfuerzo: 120 PD | Recursos: 3 | Disponibilidad: 0.8
 - Duración = $120 / (3 \times 0.8) = \mathbf{50 \text{ días calendario}}$
 - Si disponibilidad baja a 0.6 $\rightarrow 120 / (3 \times 0.6) = \mathbf{66 \text{ días (+32%)}}$

2. Factores que Impactan la Estimación: La Ecuación del Mundo Real

Modelo de Ajuste (Boehm, COCOMO II):

$$\text{Esfuerzo Ajustado} = \text{Esfuerzo Base} \times \prod(\text{Factores})$$

Nota: El símbolo $\prod$ (letra pi mayúscula) representa el **producto de múltiples factores**, de forma análoga a como el símbolo Σ representa la suma. Es decir, se multiplican todos los valores de los factores de ajuste que influyen en el esfuerzo estimado. Por ejemplo: $\prod(a, b, c) = a \times b \times c$

Esto permite capturar el efecto acumulativo de diversos aspectos (complejidad, experiencia, productividad, etc.) sobre el esfuerzo final requerido.

Factores Clave Cuantificados:

Factor	Rango Multiplicador	Origen y Ejemplo
Complejidad Técnica	0.8 (baja) - 2.5 (crítica)	Blockchain: $\times 1.8$ (IEEE TSE, 2023)
Experiencia Equipo	1.2 (junior) - 0.7 (experto)	Equipo nuevo: $\times 1.4$ (Sackman, 1968)
Productividad Histórica	$\pm 30\%$ vs referencia	Si ref=20 PD/feature: $\times 0.9$ si mejor
Disponibilidad Real	0.6 (baja) - 0.95 (alta)	Soporte concurrente: $\times 0.75$ (PMI, 2022)
Calidad Esperada	1.0 (estándar) - 1.5 (alta)	Software médico: $\times 1.3$ (FDA, 2021)
Riesgos Activos	+5–25% contingencia	APIs inestables: +15% (Stutzke, 2005)

Caso Real (Netflix, 2022):

- Esfuerzo Base: 100 PD
- Ajuste = 100×1.8 (microservicios) $\times 1.2$ (20% juniors) $\times 0.75$ (disponibilidad)
- **Resultado: 162 PD** → +62% vs inicial

3. Técnicas para Estimar el Esfuerzo

a) Juicio de Expertos (Delphi Adaptado)

Proceso desarrollado por RAND Corporation (1969):

```
graph LR
  A[Consulta anónima] --> B[Agregar resultados]
  B --> C[Discusión diferencias]
  C --> D[Re-estimación]
  D -->|3 iteraciones| E[Consenso]
```

Caso Spotify:

Migración a Kafka:

- 1.^a iteración: 120–280 PD
- Tras discusión técnica: 150–170 PD
- Consenso: 160 PD

Efectividad: Reduce sesgos en 22% (Weinberg, *Quality Software Management*, 1991)

b) Analogía (Comparación Parametrizada)

Matriz de Similitud (Capers Jones, 2007):

<i>Parámetro</i>	<i>Proyecto Ref (X)</i>	<i>Nuevo (Y)</i>	<i>Factor Similitud</i>
<i>Alcance</i>	350 PF	420 PF	$420/350 = 1.2$
<i>Complejidad</i>	Media (1.0)	Alta (1.4)	$1.4/1.0 = 1.4$
<i>Tecnología</i>	Java (1.0)	Kotlin (0.9)	$0.9/1.0 = 0.9$
<i>Esfuerzo</i>	85 PM	—	$85 \times 1.2 \times 1.4 \times 0.9 = \mathbf{128.5 PM}$

Precisión: $\pm 20\%$ si hay ≥ 3 proyectos similares (QSM, 2023)

c) Descomposición (WBS + Bottom-Up)

Regla de Carlson (NASA SEH): "Ninguna tarea $> 80h$; ninguna $< 8h$ "

Técnica PERT: Estimación = (Optimista + $4 \times$ Probable + Pesimista) / 6

Ejemplo Frontend:

<i>Tarea</i>	<i>O (PD)</i>	<i>M (PD)</i>	<i>P (PD)</i>	<i>PERT</i>
<i>Login</i>	8	12	20	$(8+48+20)/6 = \mathbf{12.7}$
<i>Perfil usuario</i>	10	15	25	$(10+60+25)/6 = \mathbf{15.8}$

- Total = 28.5 PD
- +15% integración $\rightarrow \mathbf{32.8 PD}$

d) Puntos de Función / Casos de Uso

Flujo de Conversión (IFPUG, 2023):



Productividad por Dominio (ISBSG, 2022):

<i>Tipo Sistema</i>	<i>PH/PF (Rango)</i>
<i>Financiero</i>	18–22
<i>E-commerce</i>	15–20
<i>Embedded Systems</i>	25–35

Ejemplo: 420 PF × 25 PH/PF = **10,500 PH** ≈ **1,500 PD (7h/PD)**

e) Modelos Paramétricos (COCOMO II)

Ecuación (Boehm, 2000):

$$PM = 2.94 \times (\text{Tamaño})^E \times \prod(EM_i)$$

Donde:

- Tamaño = KSLOC (miles de líneas)
- $E = 1.0997 - 0.0034 \times B$ (B = % componentes reusados)
- EM_i = Factores de esfuerzo (experiencia, herramientas, etc.)

El símbolo $\prod$ indica nuevamente un **producto multiplicativo** de todos los factores EM que modifican el esfuerzo base.

Ejemplo:

- Tamaño: 50 KSLOC
- Reuso $B = 30\% \rightarrow E \approx 0.997$
- EM promedio = 1.15

$$PM = 2.94 \times (50)^{1.05} \times 1.15 \approx \mathbf{245 PM}$$

Herramientas de Apoyo y Precisión

<i>Técnica</i>	<i>Herramienta Recomendada</i>	<i>Precisión</i>	<i>Limitaciones</i>
<i>Juicio Expertos</i>	Planning Poker	±25%	Depende de calidad expertos
<i>Analogía</i>	QSM SLIM	±20%	Requiere base histórica
<i>Descomposición</i>	Microsoft Project, ClickUp	±15%	Tarda en proyectos grandes
<i>Puntos Función</i>	Function Point Analyzer	±10%	Complejo para ágil
<i>COCOMO II</i>	COCOMO Suite	±15%	Requiere KSLOC exactos

Regla de Oro (DeMarco & Lister, *Waltzing with Bears*, 2003):

"Triangula siempre: Combina 2 técnicas. Si difieren >30%, tu incertidumbre es peligrosa"

Tabla Resumen: Selección de Técnicas por Contexto

<i>Escenario Proyecto</i>	<i>Técnica Óptima</i>	<i>Combo Recomendado</i>
<i>Requisitos inestables</i>	Juicio expertos + Ágil	Planning Poker + Velocidad histórica
<i>Proyecto repetitivo</i>	Analogía + PF	Matriz similitud + PH/PF sectorial
<i>Contrato fixed-price</i>	WBS + COCOMO	PERT + Modelo paramétrico
<i>Alta innovación técnica</i>	Delphi + Prototipado	3 iteraciones Delphi + MVP medido

Caso Boeing 787 Dreamliner:

Usaron Analogía (B777) + COCOMO II + Descomposición.

Error final: **4%** vs promedio industria **22%** (PM Journal, 2015)

Principio Final (Fred Brooks, *The Mythical Man-Month*, 1995):

"Estimación de esfuerzo ≠ manipulación matemática. Es la intersección de:

- *Ciencia (modelos)*
- *Experiencia (históricos)*
- *Sabiduría (juicio contextual)"*

Checklist de Validación

- ¿Incluiste disponibilidad real (no teórica)?
- ¿Aplicaste factores de complejidad/experiencia?
- ¿Triangulaste con ≥ 2 técnicas?
- ¿Presupuestaste $\pm 15\%$ para variaciones no lineales?
- ¿Documentaste supuestos para reestimación?

C. Estimación de Costos de Recursos

1. Costos Directos: La Columna Vertebral Financiera

Los **costos directos** son aquellos gastos que pueden asignarse de forma precisa y exclusiva al proyecto. Representan entre el **70% y 85% del presupuesto total**, según el *Project Management Institute* (2021). Una correcta estimación de estos costos es esencial para asegurar la viabilidad financiera del proyecto.

a) Recursos Humanos

Incluye salarios base, beneficios sociales (INSS, INATEC), impuestos laborales, vacaciones, aguinaldo y posibles bonificaciones por desempeño. En Nicaragua, los beneficios laborales obligatorios pueden representar entre el **30% y 45%** del salario bruto.

Fórmula: $\text{Costo} = [\text{Salario} \times (1 + \% \text{Beneficios} + \% \text{Impuestos})] \times \text{Duración}$

Ejemplo (Nicaragua):

Un desarrollador senior contratado por \$1,000 mensuales:

$$\$1,000 \times (1 + 0.30 + 0.15) = \$1,450/\text{mes}$$

"El costo real de un recurso humano técnico puede ser entre 1.4 y 1.6 veces su salario bruto" (Boehm, *Software Engineering Economics*, 1981).

b) Hardware y Software

Se consideran tanto **activos fijos (CAPEX)** —como laptops o servidores físicos depreciables en 3 años— como **gastos operativos (OPEX)** —como suscripciones a servicios en la nube, licencias mensuales o SaaS.

Fórmula para servidores físicos en la nube:

Costo Mensual = (Costo Hardware / 36) + Cargo Cloud

Ejemplo:

Un servidor físico de \$10,000 + costo de servicio AWS (\$300/mes):

$$\$10,000 / 36 + \$300 = \$278 + \$300 = \$578/\text{mes}$$

En Nicaragua, este análisis es útil para decidir entre mantener infraestructura local (on-premise) o contratar servicios como Microsoft Azure o AWS (usualmente pagados en USD).

c) Servicios Externos

Se refiere a consultores, auditorías técnicas, personal freelance, abogados de TI, etc. Aquí se aplica un **factor de riesgo contractual** para cubrir posibles ampliaciones de alcance (scope creep) o imprevistos.

Fórmula: Costo = Tarifa × Horas Estimadas × Factor de Riesgo

Ejemplo:

Consultor en ciberseguridad (\$35/hora) por 50 horas y un factor de riesgo de 1.3:

$$\$35 \times 50 \times 1.3 = \$2,275$$

2. Costos Indirectos (Overhead): La Sombra Organizacional

Son costos que **no pueden atribuirse directamente** a un proyecto, pero que son necesarios para su funcionamiento: administración, infraestructura, energía eléctrica, gestión de personal, entre otros.

Fórmula: $\text{Overhead Total} = (\text{Suma Salarios} \times 0.18) + (\text{Costos Directos} \times 0.10)$

Componentes clave:

- **Infraestructura física:** Alquiler de oficina, agua, luz, vigilancia, mobiliario. Estimado del **10-15% del total de nómina**.
- **Software corporativo:** Licencias de Microsoft 365, Zoom, plataformas ERP. En Nicaragua, puede representar **\$100-200/empleado/mes**.
- **Gestión administrativa:** Personal no técnico (recursos humanos, finanzas, dirección). Corresponde al **7-12% del costo total** del proyecto.

"Startups subestiman el overhead en un 40%, mientras que empresas consolidadas lo sobrestiman en 15%" (Gartner, 2022).

3. Técnicas para Estimación de Costos

a) Análisis de Tarifas Unitarias

Basado en valores promedio del mercado o benchmarks del sector. En Nicaragua, se puede usar información de empresas tecnológicas locales, universidades, asociaciones de software o precios internacionales ajustados al contexto nacional.

<i>Recurso</i>	<i>Rango Costo</i>	<i>Variables Críticas</i>
<i>Dev Senior Nicaragua</i>	\$12-30/hora	Lenguaje, seniority, tiempo parcial
<i>AWS EC2</i>	\$0.10-\$3.00/hora	Tipo de instancia, región
<i>Licencia Oracle</i>	\$47,500/núcleo/año	Núcleos físicos vs virtuales

b) Estimación Ascendente (Bottom-Up)

Es la técnica más precisa. Consiste en descomponer el proyecto en unidades pequeñas (paquetes de trabajo del WBS), asignar costos a cada una y luego sumarlos. Finalmente se agregan **overhead y contingencias**.

Ejemplo: Módulo de Autenticación (Login)

Frontend: $15h \times \$100/h = \$1,500$

Backend: $20h \times \$120/h = \$2,400$

QA: $8h \times \$80/h = \640

Subtotal: \$4,540

Overhead (12%): \$545

Total: \$5,085

c) Juicio de Expertos (Método Delphi)

Utiliza la experiencia colectiva de varios profesionales para definir un costo promedio. Es especialmente útil cuando se estima con incertidumbre alta (ej. nuevas tecnologías).

Experto	Estimación Licencia ERP
A	\$120,000
B	\$95,000
C	\$150,000
Consenso	\$125,000

"El consenso de cinco expertos reduce errores en un 22% frente a estimaciones individuales" (Weinberg, 1991).

d) Análisis de Reservas

Permite incluir colchones financieros para riesgos. Se divide en:

- **Riesgos conocidos (contingencia):**

Fórmula:

$\text{Reserva} = \sum (\text{Probabilidad} \times \text{Impacto})$

Ejemplo:

Probabilidad de migrar a otro servidor: 50%, impacto \$20,000 →

$\text{Reserva} = 0.5 \times \$20,000 = \$10,000$

- **Riesgos desconocidos (reserva de gestión):**
Porcentaje adicional del costo total (5–10%) para cubrir imprevistos no identificados.

4. Matriz de Selección de Técnicas

<i>Escenario</i>	<i>Técnica Óptima</i>	<i>Precisión Esperada</i>
<i>Proyectos repetitivos</i>	Bottom-Up + Tarifas unitarias	±7%
<i>Tecnologías emergentes</i>	Juicio experto + Reservas	±20%
<i>Contratos largos</i>	Modelo híbrido	±12%

5. Errores Fatales y Mitigaciones

<i>Error</i>	<i>Consecuencia</i>	<i>Mitigación</i>
<i>Ignorar escalabilidad cloud</i>	Costos exponenciales (+300%)	Simular cargas con AWS Calculator
<i>Subestimar impuestos</i>	Sanciones legales (+25%)	Consultar con contadores especializados
<i>No prever cláusulas de salida</i>	Penalización del 200% del contrato	Incluir cláusulas de terminación anticipada

6. Plantilla de Estimación Consolidada

<i>Concepto</i>	<i>Costo Base</i>	<i>Overhead</i>	<i>Contingencia</i>	<i>Total</i>
<i>Recursos Humanos</i>	\$75,000	\$15,000	\$7,500	\$97,500
<i>Hardware</i>	\$12,000	\$0	\$1,200	\$13,200
<i>Licencias</i>	\$28,000	\$2,800	\$2,240	\$33,040
<i>TOTAL</i>	\$135,000	\$17,800	\$10,940	\$163,740

Principio Final

"La estimación de costos es la termodinámica de la gestión financiera: cada dólar no presupuestado se transforma en caos operativo." (Humphrey, 1990)

Reglas de oro:

- **Triangulación:** Usa al menos dos técnicas complementarias.
- **Transparencia radical:** Documenta cada supuesto y restricción.

- **Reservas dinámicas:** Actualiza mensualmente con base en riesgos.
- **Benchmarking continuo:** Compara costos con fuentes como Gartner, ISBSG, y datos locales.

"Un presupuesto sin reservas es un proyecto sin oxígeno"
 — Tom DeMarco, *Waltzing with Bears*, 2003

D. Desafíos Clave en la Estimación de Recursos

1. Disponibilidad Real vs. Asignada: La Ilusión del 100%

Problema Fundamental:

En la práctica, la disponibilidad real de un recurso rara vez supera el **60–70%** de su asignación teórica. Barry Boehm (1981) lo cuantifica mediante la siguiente fórmula: $\text{Disponibilidad_Real} = \text{Asignación_Teórica} \times (1 - \text{Factor_Dilución})$

Donde:

- **Factor_Dilución** $\in [0.25, 0.40]$, considerando reuniones, cambios de contexto y carga burocrática.
- Ejemplo de Microsoft (2023): los desarrolladores solo dedican **5.7 h/día** a trabajo productivo.

Estrategias de Mitigación:

<i>Causa</i>	<i>Impacto Aproximado</i>	<i>Solución Basada en Evidencia</i>
Multitarea	↓ 40% productividad	Bloques de <i>Deep Work</i> (Cal Newport, 2016)
Reuniones	15–25 h/mes	Regla 30/70 (Jeff Sutherland, Scrum)
Burocracia	8–12% del tiempo	Automatización mediante CI/CD
Soporte no planificado	2–3 h/semana	Ventanas de soporte: ej. Martes/Jueves 4–5 PM

Ejemplo Visual – Disponibilidad Semanal (Dev Senior):

- Asignación Teórica: 40 h

- Uso Real:

██████████ Desarrollo (22 h)

██████ Reuniones (10 h)

████ Soporte (5 h)

██ Burocracia (3 h)

2. Curva de Aprendizaje: El Muro Invisible

Modelo Adaptado de la Curva de Olvido de Ebbinghaus:

Productividad = $1 - e^{-k \cdot t}$

Donde:

- **k = 0.05** para tecnología nueva
- **k = 0.03** para dominio nuevo
- **t** = semanas desde el inicio del trabajo

Estrategias Compensatorias:

```
def calcular_penalizacion(nuevos_miembros):  
    return 0.3 * nuevos_miembros # 30% penalización inicial
```

- **Mentoría estructurada:** Senior dedica 20% del tiempo las primeras 4 semanas (*Google, 2022*)
- **Spikes técnicos:** Dedicación del 15–20% de la capacidad en los primeros sprints

“Ignorar la curva de aprendizaje es subestimar en 30% cualquier proyecto innovador”

— *Capers Jones, 2007*

3. Variabilidad de la Productividad: El Mito del Promedio

Rango de Variación (Sackman, 1968):

Rol	Variación Individual	Variación Entre Equipos
Desarrollador	5:1	3:1
Tester	4:1	2.5:1

Tácticas de Nivelación:

- **Skill Matrix Personalizada:**

Habilidad	Nivel (1–5)	Acción	
-----	-----	-----	
React	3.2	Curso avanzado	
AWS Lambda	2.5	Certificación	

- **Estimación colaborativa:** Planning Poker con arquitectos
- **Benchmarking de productividad:** Medir la *velocity* por equipo (story points/sprint)

Gestión de Capacidad: El Arte del Equilibrio

Modelo de Optimización (Goldratt – Teoría de Restricciones):

Capacidad_Óptima = (Demanda / Disponibilidad) × (1 + Buffer)

- **Buffer** ∈ [15%, 20%] para absorber picos imprevistos

Herramientas:

- **Matriz de Asignación de Recursos:**

```
graph LR
  P1[Proyecto A] --> R1[Dev 1]
  P1 --> R2[Dev 2]
  P2[Proyecto B] --> R2
  P2 --> R3[Dev 3]
```

- **Priorización Multinivel:**

- **Crítico:** Asignación dedicada
- **Medio:** Máx. 2 proyectos por recurso
- **Bajo:** Asignación residual

Costos Ocultos: El Iceberg Financiero

Tipología y Tácticas de Mitigación:

<i>Costo Oculto</i>	<i>% del Presupuesto</i>	<i>Táctica de Visibilización</i>
<i>Onboarding</i>	3–7%	Presupuesto específico
<i>Reuniones</i>	8–12%	Límite de 25 h/semana por rol
<i>Refactoring</i>	15–25%	Incluir en la “Definición de Hecho”
<i>Deuda Técnica</i>	10–20%	Sprints dedicados trimestrales

Fórmula de Recuperación:

$$\text{Costo_Total} = \text{Costo_Estimado} \times (1 + \text{Factor_Oculto})$$

- **Factor_Oculto** ∈ [0.10 (madurez alta), 0.25 (madurez baja)]

6. Volatilidad de Precios (Cloud/SaaS): La Amenaza Silenciosa

Estrategias Antivolatilidad:

<i>Recurso</i>	<i>Técnica</i>	<i>Ejemplo Práctico</i>
<i>Cloud IaaS</i>	Reservas a 1–3 años	AWS Reserved Instances (–60%)
<i>SaaS Crítico</i>	Cláusulas contractuales	Tope anual del 7%
<i>Licencias</i>	Modelo híbrido	Kubernetes + servidores dedicados
<i>Transferencia de datos</i>	Arquitectura optimizada	CDN + Caching local

Modelo Predictivo en Python:

```
def predecir_costo_cloud(uso, proveedor):
    tendencias = {'AWS': 1.07, 'Azure': 1.05, 'GCP': 1.03}
    return uso * (tendencias[proveedor] ** 3)
```

Kit de Supervivencia para Gestores

1. Plantilla de Dilución de Tiempo

Actividad	% Tiempo	Acción Correctiva
-----	-----	-----
Reuniones	30%	Limitar a 10h/semana
Cambios de Contexto	25%	Deep Work de 2h mínimo
Burocracia	15%	Automatizar 50% procesos

2. Fórmula de Productividad Ajustada

$\text{Productividad_Real} = \text{Velocidad} \times (1 - \text{Factor_Dilución}) \times (1 - \text{Factor_Aprendizaje})$

3. Dashboard de Riesgos (Ejemplo)

- Onboarding: 7%
- Reuniones: 12%
- Refactoring: 20%

Principio Fundamental

"La estimación de recursos no busca perfección, sino convertir lo invisible en gestionable. Como dijo Deming: 'Sin datos, solo eres otra persona con una opinión'. Tu mejor herramienta es el aprendizaje continuo de desviaciones."

Checklist de Auditoría

- ¿Dispones de métricas reales de disponibilidad semanal?
- ¿Incluiste sprints para deuda técnica en los últimos 90 días?
- ¿Los contratos cloud tienen cláusulas de tope de incremento?
- ¿Utilizas skill matrix para ajustar la planificación?

Fórmulas Resumidas

1. Disponibilidad_Real = Asignación_Teórica × (1 - Factor_Dilución)
2. Productividad = $1 - e^{-k \cdot t}$
3. Capacidad_Óptima = (Demanda / Disponibilidad) × (1 + Buffer)
4. Costo_Total = Costo_Estimado × (1 + Factor_Oculto)
5. Productividad_Real = Velocidad × (1 - Factor_Dilución) × (1 - Factor_Aprendizaje)

E. Integración con la Planificación General: El Eje Operativo

La **estimación de recursos y costos** es el cimiento cuantitativo que habilita una planificación realista y efectiva. Su integración impacta cinco dimensiones críticas del proyecto, según el **PMBOK® 7ª Edición**:

flowchart TD

```
E[Estimación Recursos] --> C[Cronograma Realista]
E --> P[Presupuesto Base]
E --> R[Plan Gestión Recursos]
E --> V[Análisis Viabilidad]
E --> M[Decisiones Make-or-Buy]
```

1. Creación de Cronogramas Realistas

La relación matemática fundamental que vincula recursos con la duración del proyecto está descrita por Kerzner (2022) en *Project Management*:

$$\text{Duración} = \frac{\text{Esfuerzo (Personas-Día)}}{\text{N° Recursos} \times \text{Disponibilidad}}$$

Donde:

- **Esfuerzo (PD)**: Total de días-hombre estimados.
- **N° Recursos**: Cantidad de personas asignadas.
- **Disponibilidad**: Porcentaje efectivo de tiempo productivo (ejemplo, 0.8 = 80%).

Ejemplo Práctico aplicado a Nicaragua

Escenario	Cálculo	Duración	Consecuencia
Equipo completo (5 devs)	$200 \text{ PD} / (5 \times 0.8) = 50$ días	50 días	Factible
Recursos limitados (3)	$200 \text{ PD} / (3 \times 0.8) = 83$ días	83 días	Replanificar o ampliar alcance

Nota: En proyectos tecnológicos en Nicaragua, donde la disponibilidad puede verse afectada por factores externos (conectividad, infraestructura), es crucial validar este parámetro localmente.

Herramienta Clave:

"Simular escenarios en MS Project o Jira permite detectar cuellos de botella. Un desfase mayor al 20% entre recursos necesarios vs. disponibles obliga a renegociar fechas" (Harold Kerzner, 2022).

2. Elaboración del Presupuesto Base

El presupuesto debe integrar de manera clara todos los costos, según el estándar PMI para presupuestos:

$$\text{Presupuesto} = \underbrace{\sum \text{Costos Directos}}_{\text{Recursos}} + \overbrace{\sum \text{Overhead}}^{15-25\%} + \underbrace{\text{Reservas}}_{\substack{\text{Riesgos} \\ 5-15\%}}$$

Donde:

- **Costos Directos** incluyen salarios, licencias, servicios.
- **Overhead:** Costos indirectos administrativos y operativos.
- **Reservas:** Fondos para riesgos identificados.

Matriz de Trazabilidad:

<i>Recurso Estimado</i>	<i>Componente Presupuesto</i>	<i>Ejemplo</i>
<i>3 Devs Senior</i>	Costo humano	$3 \times \$8,000/\text{mes} \times 3 \text{ meses} = \$72,000$
<i>Licencias AWS</i>	Costo software	$\$2,300/\text{mes} \times 3 = \$6,900$
<i>QA Externo</i>	Costo terceros	\$15,000 fijo

Advertencia crítica: "Presupuestos sin desglose son incontrolables. Si no puedes rastrear un dólar hasta su recurso específico, el desvío puede superar el 30%" (Robert Glass, 2003).

3. Plan de Gestión de Recursos

El ciclo de gestión de recursos basado en estimaciones, según la metodología **PRINCE2®**, se puede representar así:

sequenceDiagram

Planificación->>+Adquisición: "3 devs para sprint 2 (skills: React/AWS)"

Adquisición->>+Asignación: "Asignar priorizando Skill Matrix"

Asignación->>+Seguimiento: "Alerta si carga >80%"

Seguimiento->>+Liberación: "Reasignar tras hito UAT"

Checklist de Implementación:

- **Picos de demanda:** Identificar fases críticas (ejemplo: +2 testers en UAT).
- **Contingencia rotación:** Mantener lista de backups con al menos 2 candidatos por rol clave.
- **Liberación gradual:** Planificar transición de recursos entre proyectos para evitar interrupciones.

4. Análisis de Viabilidad

Preguntas claves deben ser respaldadas por métricas cuantificables:

<i>Pregunta</i>	<i>Métrica</i>	<i>Umbral Crítico</i>
<i>¿Tenemos recursos?</i>	Brecha capacidad/demanda	>15% requiere acción inmediata
<i>¿Es financieramente viable?</i>	ROI Estimado	<120% considerado inviable
<i>¿Alinea con capacidades?</i>	Match de habilidades	<70% requiere capacitación

Modelo Decisorio en Python para Aprobación:

```
def aprobar_proyecto(roi, gap_recursos, riesgo_tecnico):  
    if roi >= 1.2 and gap_recursos <= 0.15 and riesgo_tecnico < 0.4:  
        return "GO"  
    elif roi >= 1.5 and riesgo_tecnico < 0.6: # Caso estratégico  
        return "GO con reservas"  
    else:  
        return "NO-GO (Replantear alcance/recursos)"
```

5. Decisiones Make-or-Buy

Basado en la matriz de Boehm (Software Economics):

<i>Factor</i>	<i>Make (Desarrollar)</i>	<i>Buy (Comprar/Subcontratar)</i>
<i>Costo Total</i>	CAPEX alto + OPEX bajo	OPEX recurrente + costos integración
<i>Control Calidad</i>	Máximo (estándares internos)	Limitado (depende proveedor)
<i>Tiempo</i>	Largo (6-24 meses)	Corto (1-3 meses)

Árbol de Decisión Validado (Capers Jones):

```
graph TD  
    A[¿Es core business?] -->|Sí| B[Make]  
    A -->|No| C[¿Existe solución madura?]  
    C -->|Sí| D[Buy]  
    C -->|No| E[¿Valor estratégico alto?]  
    E -->|Sí| B  
    E -->|No| D
```

Ejemplo real:

Spotify desarrolla internamente su stack de IA (**Make**), pero utiliza Salesforce para CRM (**Buy**).

Proceso Integrado en 4 Pasos

1. Estimación Detallada:

Técnica recomendada: *Bottom-up* por WBS con costos unitarios.

"Incluir siempre disponibilidad real (<70% en Nicaragua por factores locales) y curva de aprendizaje (+30%)."

2. Simulación de Escenarios:

Variables a considerar:

- Recursos ($\pm 20\%$)
- Disponibilidad (60-80%)
- Incrementos en precios cloud (+10% anual)

Herramientas sugeridas: Monte Carlo en @Risk o Crystal Ball.

3. Validación Cross-Funcional:

Revisar con Finanzas, Recursos Humanos y Arquitectura para validar supuestos.

Checkpoint clave: ¿Los supuestos resisten auditoría?

4. Documentación de Supuestos:

Ejemplo: "Este presupuesto asume disponibilidad 80%, rotación <10%, inflación cloud 7% anual, sin cambios regulatorios."

Señales de Alarma de Desintegración

- Diferencias >20% entre estimación técnica y financiera → Falta alineación interdisciplinaria.
- Cronogramas que ignoran disponibilidad real → Fracaso seguro en la ejecución.
- Presupuestos sin desglose por recurso → Imposibilidad de controlar desviaciones.

- Decisiones make-or-buy sin análisis TCO (Total Cost of Ownership) → Sobrecostos de hasta 40% (Gartner).

Principio de Oro

"La planificación sin estimación rigurosa de recursos es pura ficción operativa. Según el estudio CHAOS 2023, el 62% de los proyectos fracasan no por mala ejecución, sino por falsos supuestos iniciales" (Standish Group).

Checklist Final

- ¿Tu cronograma incluye disponibilidad real (no solo teórica)?
- ¿El presupuesto desglosa costos por recurso?
- ¿El plan de recursos considera contingencias para rotación?
- ¿Las decisiones make-or-buy usaron análisis TCO a 3 años?
- ¿Documentaste todos los supuestos y restricciones?

"En la guerra de los proyectos, los presupuestos son balas. Quien dispara sin estimar recursos, solo hace ruido." (Adaptado de Sun Tzu).

Evaluación: Gestión de Recursos en Proyectos Tecnológicos

Instrucciones:

1. Responda todas las preguntas de forma clara, ordenada y concisa.
2. Justifique cada respuesta con conceptos, fórmulas o modelos del material estudiado.
3. Use calculadora básica si es necesario. No se permiten dispositivos electrónicos.
4. Escriba con letra legible.

Sección 1: Análisis Conceptual

1. Explique la **Ley de Brooks** ("Añadir más programadores a un proyecto retrasado solo lo retrasará más") desde dos perspectivas:
 - a) Matemática (fórmula de comunicaciones)
 - b) Psicosocial (impacto en equipos)
2. Describa TRES costos ocultos en la estimación de recursos físicos/logísticos según Forrester (2022) y cómo afectan al presupuesto.

Sección 2: Aplicación de Modelos

Caso de estudio:

Un proyecto de desarrollo requiere:

- **Alcance:** 180 Puntos de Función (PF)
- **Equipo:**
 - 1 Arquitecto (tasa \$70/hora, 90% disponibilidad)
 - 3 Desarrolladores (2 Senior factor 0.7, 1 Junior factor 1.5; tasa promedio \$50/hora)
- **Complejidad:** Alta (factor 1.6)
- **Velocidad histórica del equipo:** 22 PF/sprint

Tareas:

1. Calcule el **esfuerzo total ajustado** en Personas-Día (PD).
2. Determine la **duración en sprints** (2 semanas/sprint).
3. Estime el **costo total de recursos humanos** (en USD) incluyendo beneficios sociales (30%).

Sección 3: Gestión de Riesgos

Escenario:

Usted lidera un proyecto que usará:

- **Infraestructura cloud AWS** (servicios EC2 y S3)
- **Licencias Oracle** (núcleos físicos: 16)
- **Equipo externo** (modelo Time & Material)

Identifique:

1. DOS riesgos específicos de **cada tipo de recurso** (hardware, software, humanos).
2. Una **estrategia de mitigación** para cada riesgo.
3. Calcule el **costo anual de licencias Oracle** y proponga una alternativa para reducir costos.

Sección 4: Toma de Decisiones

Contexto:

Un cliente solicita desarrollar un módulo de inteligencia artificial. Existen dos opciones:

- **MAKE:** Desarrollo interno. Esfuerzo estimado: 120 PD, costo \$95,000.
- **BUY:** SaaS de terceros. Costo: \$3,000/mes + \$15,000 implementación.

Analice:

1. **Tres criterios clave** (no financieros) para elegir entre MAKE o BUY.
2. **Cálculo de TCO** (Total Cost of Ownership) a 3 años para ambas opciones.

3. Recomendación final justificada.

Criterios de Evaluación

<i>Criterio</i>	<i>%</i>	<i>Indicadores de Desempeño</i>
<i>Dominio Conceptual</i>	25%	• Precisión en definiciones (Ley de Brooks, costos ocultos) • Uso correcto de terminología técnica • Relación teoría-caso real
<i>Aplicación de Modelos</i>	30%	• Correcto uso de fórmulas (Boehm, ajustes por experiencia) • Cálculos coherentes con unidades • Contextualización de variables
<i>Gestión de Riesgos</i>	25%	• Identificación relevante de riesgos por recurso • Viabilidad de estrategias de mitigación • Creatividad en alternativas de reducción de costos
<i>Toma de Decisiones</i>	15%	• Criterios no financieros válidos • Cálculo completo de TCO • Coherencia en recomendación
<i>Claridad y Estructura</i>	5%	• Respuestas organizadas y legibles • Síntesis efectiva • Ortografía y redacción adecuadas

Observaciones Finales:

- **No se aceptan respuestas sin justificación conceptual.**
- **En cálculos:** Mostrar fórmulas y pasos intermedios.
- **Párrafos extensos sin síntesis serán penalizados en claridad.**

Técnicas de descomposición

A. Concepto Fundamental

La descomposición en ingeniería del software consiste en fragmentar un sistema complejo en partes más pequeñas, comprensibles y gestionables. Esta filosofía puede resumirse en la fórmula conceptual:

$$\text{Problema Grande} = \Sigma (\text{Partes Pequeñas}) + \text{Interacciones}$$

Esta técnica, ampliamente documentada desde las contribuciones de Barry Boehm en *Software Engineering Economics* (1981), representa un pilar de la planificación efectiva, especialmente en proyectos de desarrollo de sistemas complejos. Su propósito es reducir la incertidumbre, mejorar la precisión de estimaciones y permitir un control granular del proyecto.

Beneficios fundamentales:

- **Mayor precisión en estimaciones:** Según datos de QSM Associates (2023), las estimaciones realizadas a partir de componentes individuales logran una reducción de error desde un rango típico de $\pm 30\text{-}50\%$ hasta niveles mucho más aceptables de $\pm 5\text{-}15\%$.
- **Visibilidad anticipada de riesgos:** Fragmentar el sistema desde etapas tempranas permite identificar componentes de alto riesgo, como aquellos que requieren integración con sistemas legados, APIs de terceros o tecnologías poco conocidas.
- **Control más granular:** Permite realizar ajustes en paquetes de trabajo específicos sin necesidad de reconfigurar todo el cronograma, lo cual puede representar una mejora en eficiencia de hasta un 20% en tiempo de gestión, según prácticas observadas por equipos Scrum en entornos industriales.

Frederick Brooks, en su icónica obra *The Mythical Man-Month*, sintetizó esta idea de forma poderosa:

"Estimamos átomos, no moléculas. La suma de verdades pequeñas es más exacta que una gran aproximación."

B. Técnicas Principales

1. Descomposición Basada en el Alcance (Work Breakdown Structure - WBS o EDT)

La Estructura de Desglose del Trabajo (EDT), también conocida como WBS por sus siglas en inglés, se basa en representar jerárquicamente el alcance total del proyecto dividiéndolo en entregables y paquetes de trabajo.

Estructura jerárquica típica:

Proyecto

├─ Entregable 1

│ └─ Componente 1.1

│ │ └─ Paquete de Trabajo 1.1.1 (12 días-persona)

│ │ └─ Paquete de Trabajo 1.1.2 (18 días-persona)

└─ Entregable 2

Reglas clave:

- **Regla del 8/80:** Cada paquete de trabajo debe tener una duración estimada entre 8 y 80 horas, para mantenerlo manejable (recomendación del *NASA Systems Engineering Handbook*).
- **Principio MECE:** Las tareas deben ser Mutuamente Excluyentes y Colectivamente Exhaustivas, es decir, no solaparse entre sí y cubrir el 100% del trabajo previsto (principio promovido por la firma McKinsey).
- **Número óptimo de niveles:** De 4 a 6 niveles jerárquicos, según lo recomendado por el *PMBOK® Guide* del PMI.

Ejemplo desarrollado:

Supongamos que estamos desarrollando un sistema de comercio electrónico para una cooperativa agrícola en Nicaragua que busca vender productos como café y cacao en línea.

Nivel 1: Sistema de E-commerce

Nivel 2: Backend

Nivel 3: Módulo de Pedidos

Nivel 4:

- API de creación de pedido → estimado: 12 días-persona
- API de procesamiento de pagos → estimado: 18 días-persona

Ventaja clave: Esta técnica es ideal cuando el contrato es de precio fijo (*fixed-price*), ya que permite congelar el alcance y detallar las entregas con precisión.

2. Descomposición Basada en Funcionalidad (Análisis de Puntos de Función - APF)

La técnica de puntos de función, promovida por IFPUG (International Function Point Users Group), busca cuantificar la funcionalidad que el usuario percibe del sistema. Es especialmente útil en sistemas orientados a procesamiento transaccional.

Proceso paso a paso:

1. Clasificación de componentes funcionales:

- ALI (Archivos Lógicos Internos): 3 (por ejemplo, tabla de usuarios, productos y órdenes).
- EE (Entradas Externas): 12 (como formularios de registro, inicio de sesión, agregar al carrito, realizar pago, etc.).

2. Asignación de complejidad:

Tipo Baja Media Alta

ALI 7 10 15

EE 3 4 6

3. Supongamos que los ALI son de complejidad media (10) y los EE de complejidad media (4).

4. Cálculo de Puntos de Función No Ajustados (PFNA):

$$\text{PFNA} = (3 \times 10) + (12 \times 4)$$

$$\text{PFNA} = 30 + 48 = 78$$

5. **Aplicación de factores de ajuste:** Supongamos que la suma de los 14 factores de ajuste da 28.

$$\text{PFA} = 78 \times [0.65 + (0.01 \times 28)]$$

$$\text{PFA} = 78 \times 0.93 = 72.5 \text{ puntos de función}$$

6. Conversión a esfuerzo (basado en datos históricos):

$$\text{Esfuerzo} = 72.5 \times 20 \text{ horas/punto}$$

$$\text{Esfuerzo} = 1,450 \text{ horas} \approx 207 \text{ días-persona}$$

Aplicación típica: Desarrollo de plataformas bancarias, sistemas de facturación, nómina o ERP en universidades nicaragüenses.

3. Descomposición Basada en Actividades

Este enfoque se centra en las tareas necesarias para completar cada componente del proyecto. Es común en metodologías ágiles y de desarrollo iterativo, donde la planificación se organiza por actividades y tareas.

Ejemplo: Supongamos que se está desarrollando el módulo de inicio de sesión (login) de un sistema de gestión académica para una universidad en Nicaragua.

Componente: Módulo Login

├─ Actividad: Diseño

| └─ Tarea: Diseño de la base de datos (modelado de usuario, roles, tokens)

| └─ Tarea: Diseño de la interfaz API (especificación OpenAPI)

├─ Actividad: Codificación

├─ Actividad: Pruebas unitarias y funcionales

Estimaciones (usando técnicas mixtas):

- Diseño: Técnica de 3 puntos → (Pesimista: 3, Optimista: 2, Más probable: 4)
Esfuerzo = $(3 + 4 \times 2 + 2) / 6 = 3.5$ días-persona
- Codificación: Por analogía con módulos similares → 8.2 días-persona
- Pruebas: 50% del esfuerzo de codificación → 4.1 días-persona

Total estimado: 15.8 días-persona

Aplicación óptima: Equipos ágiles que trabajan con herramientas como Jira o Azure DevOps, permitiendo planificación y seguimiento detallado por tareas.

C. Comparativa de Técnicas

<i>Criterio</i>	<i>WBS</i>	<i>Puntos de Función</i>	<i>Actividades</i>
<i>Precisión</i>	±10%	±15%	±20%
<i>Momento de uso</i>	Inicio del proyecto (con alcance definido)	Durante análisis de requisitos	Durante planificación de sprints
<i>Complejidad</i>	Media (estructuración jerárquica)	Alta (requiere formación especializada)	Baja (flexible y adaptable)
<i>Mejor escenario</i>	Infraestructura física o contractual	Sistemas financieros y transaccionales	Startups y MVP ágiles

Errores Comunes y Soluciones

<i>Error</i>	<i>Consecuencia</i>	<i>Mitigación</i>
<i>Descomposición incompleta</i>	Omisión de tareas críticas	Validar con checklist MECE
<i>Ignorar interfaces</i>	Subestimación de esfuerzo en 15%	Agregar explícitamente los "costos de conexión"
<i>Sobredescomposición</i>	Parálisis por análisis	Limitar a 6 niveles jerárquicos (ISO 21500)
<i>Homogeneidad de complejidad</i>	Falta de precisión en tareas clave	Usar estimaciones PERT con tres valores

Plantilla de Descomposición Óptima

<i>Nivel</i>	<i>Tipo</i>	<i>Elemento</i>	<i>Técnica</i>	<i>Esfuerzo</i>
1	WBS	Módulo de Pagos		40 PD
2	Actividades	Diseño API	Técnica de 3 puntos	8 PD
2	Actividades	Codificación	Analogía	18 PD
3	Funcionalidad	API Crear Pago	Puntos de Función	7.2 PD

Conclusión

La descomposición no es un arte improvisado; es una disciplina basada en la aplicación de principios científicos. Herbert A. Simon, en *The Sciences of the Artificial* (1969), argumentó:

"La complejidad se domina mediante jerarquías".

Por ello, no se debe escoger la técnica de descomposición por familiaridad o conveniencia, sino por la naturaleza y estructura del sistema que se desea abordar. La elección de la técnica correcta es la primera decisión que marca la diferencia entre un proyecto exitoso y uno caótico.

Checklist de Implementación

1. Definir el número máximo de niveles (no más de seis).
2. Asignar la técnica más adecuada según el tipo de componente (WBS, PF, Actividades).
3. Incorporar al menos un 15% de esfuerzo adicional para el tratamiento de interfaces e integraciones.
4. Validar la cobertura total mediante los principios MECE.
5. Documentar todos los supuestos técnicos, restricciones y dependencias por cada paquete de trabajo.

Evaluación: Técnicas de Descomposición en Ingeniería del Software

Sección A: Conceptuales (Explicación y Fundamentación)

1. Explique detalladamente el concepto de descomposición en ingeniería del software. Incluya:

- Objetivo de aplicar esta técnica.
- Referencias teóricas o autores clave que la respaldan.
- Beneficios principales que se derivan de su aplicación en proyectos complejos.

2. Analice la frase de Frederick Brooks:

"Estimamos átomos, no moléculas. La suma de verdades pequeñas es más exacta que una gran aproximación."

¿Qué relación tiene esta frase con la precisión en las estimaciones cuando se aplica la descomposición? Fundamente su respuesta con un ejemplo contextualizado al desarrollo de software en Nicaragua.

Sección B: Aplicación Técnica

3. A partir del siguiente caso, elabore una **estructura de descomposición basada en WBS** de al menos 4 niveles:

Una cooperativa pesquera en Nicaragua desea desarrollar un sistema web para registrar la captura diaria, exportaciones y control de refrigeración. El sistema debe tener módulos de registro, seguimiento de temperatura, y generación de reportes para exportación.

Incluya estimaciones en días-persona para al menos dos paquetes de trabajo del nivel más bajo. Justifique si aplica la regla del 8/80, el principio MECE y el número recomendado de niveles.

4. Para el sistema anterior, realice una **estimación funcional utilizando la técnica de puntos de función (APF)**:

- Clasifique al menos 2 tipos de componentes funcionales (por ejemplo, ALI, EE).
- Asigne niveles de complejidad.
- Calcule los puntos de función no ajustados (PFNA).
- Aplique un factor de ajuste hipotético.
- Estime el esfuerzo total en horas y días-persona.

5. Describa cómo aplicaría una **descomposición basada en actividades** para desarrollar el módulo de generación de reportes del sistema. Incluya actividades, tareas y una estimación por la técnica de los tres puntos. Justifique por qué esta técnica es adecuada para esta parte del sistema.

Sección C: Evaluación Crítica y Comparativa

6. Compare las tres técnicas de descomposición presentadas (WBS, Puntos de Función, Actividades) con base en:

- Precisión estimada.
- Momento ideal de aplicación.
- Nivel de complejidad.
- Contexto o escenario ideal de uso.

Utilice una tabla comparativa para estructurar su respuesta.

7. Analice dos errores comunes que pueden surgir durante la descomposición de un sistema. Explique sus consecuencias y proponga una estrategia de mitigación concreta basada en los principios estudiados.

Sección D: Síntesis y Aplicación Práctica

8. Suponga que está liderando un proyecto para desarrollar un sistema de gestión académica para una universidad en Nicaragua. Con base en lo aprendido:

- Elabore una propuesta de plantilla de descomposición óptima para uno de los módulos (por ejemplo, control de matrícula, asignación de docentes, seguimiento de asistencia).
- Indique qué técnica usará para cada nivel de descomposición.
- Justifique las decisiones tomadas y los criterios utilizados para seleccionar las técnicas.

Criterios de Evaluación

<i>Criterio</i>	<i>%</i>
<i>Dominio conceptual sobre descomposición y sus fundamentos teóricos</i>	20%
<i>Aplicación correcta de técnicas WBS, Puntos de Función y Actividades</i>	30%
<i>Claridad y profundidad en la justificación técnica de estimaciones</i>	15%
<i>Capacidad de análisis comparativo y crítico sobre métodos</i>	15%
<i>Aplicación contextualizada al entorno nicaragüense</i>	10%
<i>Presentación estructurada, coherencia y uso adecuado del lenguaje técnico</i>	10%

Modelos de estimación empíricos

A. Concepto Fundamental

Los modelos de estimación empíricos son herramientas matemáticas que permiten traducir las características de un proyecto de software (como tamaño, complejidad y entorno de desarrollo) en valores numéricos aproximados de esfuerzo, duración o costo. Su utilidad es fundamental para planificar recursos, reducir incertidumbre y controlar desviaciones a lo largo del ciclo de vida del software.

El núcleo matemático de esta clase de modelos puede representarse como:

$$\hat{E} = f(\text{Tamaño, Complejidad, Entorno}) + \varepsilon$$

Donde:

- $\hat{E}$ representa el esfuerzo estimado (por lo general, en persona-mes).
- ε es el error residual que disminuye a medida que se acumulan datos históricos (Boehm, 1981).

Como expresó Capers Jones en *Applied Software Measurement*: "El futuro del software está oculto en los patrones del pasado. Sin datos históricos, las estimaciones son apuestas disfrazadas".

Los modelos empíricos se apoyan en dos principios científicos clave:

- **Ley de los Rendimientos Decrecientes:** Aumentar el número de personas no implica reducir proporcionalmente el tiempo de desarrollo (Brooks, 1975).
- **Economías de Escala:** Proyectos grandes (por ejemplo, mayores a 100,000 líneas de código) tienden a tener menor costo por línea, debido a la reutilización, automatización y sinergias internas (COCOMO II).

B. Modelos Clave

1. Modelo COCOMO (Constructive Cost Model)

Propuesto por Barry Boehm en 1981, COCOMO es uno de los modelos más influyentes para estimar esfuerzo en el desarrollo de software. Posee tres variantes:

- **COCOMO Básico:** Usa solo el tamaño del software, expresado en miles de líneas de código fuente (KDSI), y clasifica el proyecto como:
 - *Orgánico*: proyectos pequeños con equipos cohesionados.
 - *Semi-acoplado*: tamaño mediano con requisitos moderadamente estables.
 - *Empotrado*: sistemas con restricciones estrictas (e.g. tiempo real).
- **COCOMO Intermedio:** Introduce hasta 15 multiplicadores de costo (cost drivers), como la confiabilidad requerida, experiencia del equipo, herramientas utilizadas, entre otros.
- **COCOMO Detallado:** Divide el esfuerzo total entre fases del ciclo de vida (análisis, diseño, codificación, pruebas).

Ejemplo aplicado (contexto nicaragüense):

Supongamos un proyecto de software de facturación para una cooperativa agrícola en Nicaragua. Si se estima un tamaño de 50 KDSI:

<i>Tipo de proyecto</i>	<i>Coeficientes (a, b)</i>	<i>Estimación (persona-mes)</i>
<i>Orgánico</i>	a = 2.4, b = 1.05	143 PM
<i>Semi-acoplado</i>	a = 3.0, b = 1.12	198 PM
<i>Empotrado</i>	a = 3.6, b = 1.20	272 PM

Este modelo ayudó a reducir errores de estimación en proyectos del Departamento de Defensa de EE.UU. del 300% al 30% (Boehm, 1981).

2. Modelo COCOMO II

Es una evolución modular del modelo original, adaptado a los entornos modernos de desarrollo, incluyendo la reutilización, la programación orientada a objetos y las arquitecturas distribuidas.

Se divide en tres etapas de uso:

- **Composición de Aplicaciones:** útil cuando el sistema se construye ensamblando componentes existentes.
- **Diseño Temprano:** cuando se tiene una idea general del sistema, pero no todos los requisitos definidos.
- **Post-Arquitectura:** se aplica cuando el diseño técnico ya está establecido y se conoce el entorno de desarrollo.

La ecuación central se expresa así:

$E = 2.94 \times (\text{Tamaño efectivo})^E \times \text{Producto de los multiplicadores de esfuerzo}$

- *Tamaño efectivo* = KDSI ajustado por reutilización.
- *Exponentes de escala* = reflejan factores como precedencia, madurez del proceso, y cohesión del equipo.
- *Multiplicadores de esfuerzo (EM)* = hasta 17 factores relacionados con requerimientos, experiencia, herramientas y otros.

Ejemplo práctico:

Un sistema contable para una municipalidad con:

- 30 KDSI de código,
- 40% de código reutilizable,
- exponentes de escala que suman 3.2 (lo que implica un exponente $E = 0.91 + 0.032$),
- multiplicadores de esfuerzo promedio de 1.10.

Tamaño efectivo = $30 \times (1 - 0.40) = 18$ KDSI

Entonces:

Esfuerzo estimado $\approx 2.94 \times 18^{0.942} \times 1.10 \approx 98$ persona-mes

3. Otros Modelos Relevantes

<i>Modelo</i>	<i>Fundamento</i>	<i>Fortalezas</i>	<i>Limitaciones</i>
<i>Puntos de Caso de Uso</i>	Basado en diagramas UML (Gustav Karner)	Fácil implementación en etapas tempranas	No contempla requerimientos no funcionales
<i>SEER-SEM</i>	Basado en ecuaciones diferenciales (Galarath Inc.)	Alta precisión en proyectos críticos	Curva de aprendizaje muy empinada
<i>SLIM (Putnam)</i>	Usa la curva de Rayleigh y fórmula de productividad	Gestión de riesgos avanzada	Requiere duración exacta del proyecto
<i>Machine Learning</i>	Algoritmos entrenados con datos reales	Alta adaptabilidad y mejora continua	Requiere bases de datos robustas y de calidad

Nota sobre SLIM:

La fórmula de Putnam se basa en:

$$K = L^3 / (C_k^3 \times t_d^4)$$

Donde K es el esfuerzo total, L el tamaño, C_k el factor ambiental, y t_d el tiempo de desarrollo.

Ejemplo: en una startup nicaragüense con entorno ágil, el factor C_k puede rondar 8000, mientras que en instituciones públicas con alta burocracia, puede ser de 1250.

C. Ventajas y Limitaciones de los Modelos Empíricos

Área	Ventajas	Limitaciones	Mitigación sugerida
Objetividad	Reduce sesgos del juicio humano	Ignora dinámicas interpersonales	Combinar con revisión por expertos
Repetibilidad	Estimaciones consistentes	Depende de datos históricos fiables	Auditorías periódicas y calibración
Velocidad	Cálculos rápidos	Requiere calibración inicial rigurosa	Uso de plantillas precalibradas
Precisión	±15% en entornos estables	Puede fallar en innovación radical	Aplicar solo a componentes predecibles

D. Guía de Implementación Práctica

1. Calibrar el Modelo

- Analizar proyectos previos y comparar estimaciones vs. resultados reales.
- Calcular un factor de corrección (ej. si la media de subestimación fue del 20%, aplicar un multiplicador de 1.20 en futuros proyectos).

2. Seleccionar el Modelo Según el Escenario

Escenario	Modelo sugerido	Ejemplo real
Sistema bancario tradicional	COCOMO II	Core bancario en bancos rurales
Sistema de defensa o misión crítica	SEER-SEM	Sistema de control en aeropuertos
Aplicación móvil en entorno ágil	Machine Learning	Plataforma de educación móvil en zona rural
MVP en fase inicial	Puntos de Caso de Uso	Sistema de gestión de citas para un centro de salud local

3. Validar de Forma Continua

- Comparar estimación vs. esfuerzo real en cada iteración o sprint.
- Si la desviación supera el ±15%, recalibrar y actualizar la base de datos histórica.
- Registrar supuestos y condiciones de cada estimación para futuras referencias.

E. Conclusión

Los modelos empíricos no son oráculos infalibles. Son instrumentos que transforman datos pasados en predicciones verificables, pero siempre deben complementarse con juicio experto y conocimiento contextual. Como bien lo resumió Barry Boehm: "Los modelos empíricos son termómetros, no doctores".

Y, recordando a Lord Kelvin: "Lo que no se define no se puede medir; lo que no se mide no se puede mejorar".

Evaluación Sumativa – Ingeniería del Software II

Tema: Modelos de estimación empíricos

Duración: 90 minutos

Modalidad: Escrita

Instrucciones generales:

- Lee cuidadosamente cada pregunta antes de responder.
- Justifica todas tus respuestas de forma clara.
- No se permite el uso de computadoras o calculadoras programables.
- Solo se evaluarán los criterios establecidos

Parte I: Conocimiento y comprensión

1. **Defina brevemente qué es un modelo de estimación empírico en el contexto del desarrollo de software.**

Incluya al menos tres características que lo diferencien de un modelo de estimación heurístico o experto.

2. **Explique qué representa cada uno de los elementos de la siguiente fórmula:**

$$E^{\wedge}=f(\text{tamaño, complejidad, entorno})+\varepsilon$$

3. **Relacione los siguientes conceptos con su definición correspondiente:** (Escriba la letra que corresponda)

a. Ley de los Rendimientos Decrecientes

- b. Economías de Escala
- c. Error Residual
- d. Reutilización de Código
 - () La proporción de esfuerzo no explicada por el modelo, que tiende a disminuir con más datos históricos.
 - () Beneficio de menor costo por unidad a medida que aumenta el tamaño del proyecto.
 - () No se puede compensar tiempo agregando más programadores en fases tardías.
 - () Reducción del tamaño efectivo del software estimado.

Parte II: Aplicación de modelos

4. Caso práctico:

Una cooperativa agrícola en Estelí desea desarrollar un sistema de gestión de insumos con un tamaño estimado de 60 KDSI. El proyecto será desarrollado por un equipo local con experiencia moderada.

Con base en el modelo **COCOMO Básico**, complete la siguiente tabla con el esfuerzo estimado (persona-mes) usando los coeficientes de cada tipo de proyecto:

<i>Tipo de proyecto</i>	<i>Coeficiente a</i>	<i>Coeficiente b</i>	<i>Fórmula</i>	<i>Resultado (PM)</i>
<i>Orgánico</i>	2.4	1.05	$E = a \times KDSI^b$	
<i>Semi-acoplado</i>	3.0	1.12	$E = a \times KDSI^b$	
<i>Empotrado</i>	3.6	1.20	$E = a \times KDSI^b$	

5. Muestre los cálculos matemáticos para **al menos un caso**.

6. Explique en qué momento del ciclo de vida del software es más adecuado usar cada una de las siguientes fases de COCOMO II y justifique con ejemplos nicaragüenses concretos:

- Composición de aplicaciones
- Diseño temprano
- Post-arquitectura

Parte III: Análisis crítico y reflexión contextual

6. **Analice críticamente las limitaciones de aplicar modelos como COCOMO o SLIM en proyectos tecnológicos desarrollados por startups en Nicaragua.**

Considere factores como disponibilidad de datos, tamaño de los equipos, infraestructura y madurez del proceso.

7. **Considere un escenario en el que una municipalidad desea implementar un sistema de recolección y análisis de datos ambientales. ¿Qué modelo empírico recomendaría para estimar el esfuerzo y por qué? Argumente al menos tres razones en función del contexto nicaragüense.**
8. **Verdadero o Falso:** Justifique su respuesta (no basta con marcar).
- a. "El modelo de Puntos de Caso de Uso es adecuado para proyectos que ya han pasado por la fase de implementación."
 - b. "COCOMO II permite incluir aspectos como la reutilización de componentes y la experiencia del equipo."

Parte IV: Investigación guiada (tarea complementaria)

Investigue y responda en casa (entregar en la próxima clase):

9. Busque un artículo o informe académico sobre la aplicación de modelos empíricos de estimación en Latinoamérica o Centroamérica. Resuma:
- Tipo de modelo utilizado
 - Contexto del proyecto
 - Principales hallazgos y dificultades reportadas
 - Aplicabilidad de los resultados al contexto nicaragüense

Criterios de Evaluación

<i>Criterio</i>	<i>Descripción</i>
Comprensión del concepto	Reconoce adecuadamente el propósito, ventajas y límites de los modelos empíricos.
Aplicación matemática	Aplica correctamente las fórmulas y muestra procedimientos claros.
Análisis contextual	Relaciona los conceptos con ejemplos del entorno nacional.
Argumentación crítica	Justifica las respuestas con razonamiento técnico y evidencia.
Claridad y presentación	Redacción ordenada, ideas claras y vocabulario técnico adecuado.

Bibliografía y Webgrafía

1. **Boehm, B. W.** (1981). *Software engineering economics*. Prentice Hall.
2. **Cohn, M.** (2005). *Agile estimating and planning*. Prentice Hall.
3. **Gartner.** (2023). *Forecast analysis: Cloud infrastructure services, worldwide*.
4. **International Function Point Users Group.** (2023). *Function point counting practices manual (v4.3.1)*.
5. **Jones, C.** (2007). *Estimating software costs: Bringing realism to estimating* (2nd ed.). McGraw-Hill.
6. **Jørgensen, M.** (2007). Forecasting of software development work effort: Evidence on expert judgment and formal models. *International Journal of Forecasting*, *23*(3), 449–462. <https://doi.org/10.1016/j.ijforecast.2007.05.008>
7. **McConnell, S.** (2006). *Software estimation: Demystifying the black art*. Microsoft Press.
8. **Project Management Institute.** (2021). *A guide to the project management body of knowledge (PMBOK® Guide)* (7th ed.).
9. **Sackman, H.** (1968). Experimental comparison of time-sharing and batch processing. *Communications of the ACM*, *11*(7), 461–467. <https://doi.org/10.1145/363397.363744>

10. **Salazar, G.** (2009). Estimación de proyectos de software: Un caso práctico. *Revista de Ingeniería de Sistemas*, *15*(1), 45-60. <https://doi.org/10.15517/ris.v15i1.3098>
11. **Standish Group.** (2018). *CHAOS report 2018: Decision latency theory*.
12. **Weinberg, G.** (1971). *The psychology of computer programming*. Dorset House.

Glosario

- **Alcance (Scope):** Conjunto de funcionalidades, características y entregables acordados para un proyecto de software.
- **Análisis de Puntos de Función (APF):** Técnica para medir el tamaño funcional del software desde la perspectiva del usuario.
- **Backlog:** Lista priorizada de requisitos o tareas pendientes en metodologías ágiles.
- **Benchmarking:** Proceso de comparar métricas propias con estándares de la industria o proyectos similares.
- **Burnout:** Agotamiento físico o mental del equipo debido a sobrecarga sostenida de trabajo.
- **CAPEX (Capital Expenditure):** Inversión en activos tangibles (hardware, infraestructura) con vida útil prolongada.
- **COCOMO II:** Modelo empírico evolucionado para estimación de software que considera reutilización, modernas prácticas de desarrollo y ajustes dinámicos.
- **Contingencias:** Reservas de tiempo, costo o recursos para mitigar riesgos identificados.
- **CPI (Índice de Desempeño de Costos):** Métrica que compara el valor ganado (EV) con el costo real (AC) para evaluar eficiencia económica.
- **Deuda Técnica:** Costo implícito de retrabajo futuro por atajos tomados durante el desarrollo.
- **Descomposición:** Técnica de dividir un sistema complejo en componentes manejables (ej. WBS).
- **Earned Value Management (EVM):** Metodología para medir avance, desempeño y pronóstico de proyectos integrando alcance, cronograma y costos.
- **Estimación Análoga:** Técnica basada en comparación con proyectos históricos similares.

- **Estimación por Tres Puntos:** Método que usa valores optimista (O), probable (M) y pesimista (P) para calcular esfuerzo: $E=O+4M+P/6$
- **Esfuerzo:** Cantidad de trabajo requerido, medido en horas-persona (HH), días-persona (PD) o meses-persona (PM).
- **Factor de Dilución:** Porcentaje de tiempo perdido en actividades no productivas (reuniones, burocracia).
- **Fixed Price:** Contrato donde el precio se fija independientemente de los recursos utilizados.
- **Function Point (Punto de Función):** Unidad de medida que cuantifica la funcionalidad entregada al usuario.
- **Gestión de Riesgos:** Proceso de identificación, análisis y respuesta a amenazas que impactan el proyecto.
- **Holgura (Slack):** Tiempo adicional disponible en una tarea sin afectar la ruta crítica.
- **KDSI (Kilo Delivered Source Instructions):** Miles de líneas de código fuente entregadas; métrica de tamaño en COCOMO.
- **Ley de Brooks:** Principio que señala que añadir personal a un proyecto retrasado lo retrasa más debido a costos de comunicación.
- **Make-or-Buy:** Decisión estratégica entre desarrollar internamente o adquirir una solución externa.
- **Monte Carlo:** Técnica de simulación probabilística para modelar incertidumbre en estimaciones.
- **MoSCoW:** Técnica de priorización (Must, Should, Could, Won't) para gestionar alcance.
- **MVP (Minimum Viable Product):** Versión mínima funcional de un producto para validar su viabilidad.
- **OPEX (Operational Expenditure):** Gastos operativos recurrentes (licencias SaaS, cloud, mantenimiento).
- **Overhead:** Costos indirectos asociados a la operación del proyecto (administración, infraestructura).

- **Planning Poker:** Técnica colaborativa ágil para estimar esfuerzo mediante consenso grupal.
- **PERT (Program Evaluation and Review Technique):** Método estadístico para estimar duraciones usando escenarios optimista, probable y pesimista.
- **Productividad:** Relación entre output generado (ej. puntos de función) y recursos invertidos (ej. horas).
- **ROI (Retorno de Inversión):** Beneficio económico esperado vs. costo del proyecto.
- **Scope Creep:** Expansión no controlada del alcance durante el desarrollo.
- **SLA (Service Level Agreement):** Acuerdo formal sobre niveles de servicio (disponibilidad, rendimiento).
- **SPI (Índice de Desempeño del Cronograma):** Métrica que compara valor ganado (EV) con valor planeado (PV) para evaluar avance temporal.
- **TCO (Total Cost of Ownership):** Costo total de adquirir, operar y mantener un activo durante su vida útil.
- **Time & Materials:** Modelo contractual donde se paga por recursos utilizados (tiempo + materiales).
- **Valor Ganado (Earned Value - EV):** Valor del trabajo realmente completado en un momento dado.
- **Velocidad (Velocity):** Métrica ágil que mide la cantidad de trabajo completado por sprint (ej. en story points).
- **WBS (Work Breakdown Structure):** Estructura jerárquica que descompone el proyecto en entregables y paquetes de trabajo.